سُبْحَانَكَ لاَ عِلْمَ لَنَا إِلاَّ مَا عَلَّمْتَنَا ۖ إِنَّكَ أَنْتَ الْعَلِيمُ الْحَكِيمُ

Dedications

In the name of Allah, the Most Gracious, the Most Merciful

To our beloved parents,

whose unwavering support, endless sacrifices, and constant prayers
have been the foundation of our success.

To our teachers and mentors,
who guided us with knowledge and wisdom
throughout our academic journey.

To the team at Novation City,
who welcomed us with open arms and provided
invaluable learning opportunities.

To our families and friends,
whose encouragement and understanding
made this achievement possible.

With deep gratitude and appreciation.

Acknowledgments

I would like to express my sincere gratitude to all those who contributed to the success of this professional internship experience, including academic supervisors, industry mentors, and institutional staff whose guidance, feedback, and practical support were essential to achieving the project objectives.

First, I extend my deepest appreciation to my academic supervisor, **Mr. Ahmed Bin Ayed**, for their invaluable guidance and continuous support throughout this project. Their expertise was instrumental in ensuring academic rigor and alignment with university requirements.

I am profoundly grateful to the Novation City team who welcomed me into their innovative ecosystem. Special thanks to my professional supervisor, **Mr. Mohamed Chrifa**, for their professional insights and technical guidance in Industry 4.0 implementation and digital transformation consulting.

My sincere appreciation goes to Novation City competitiveness cluster for entrusting me with developing the **IoT-REAP (Internet of Things - Remote Equipment Access Platform)**. Building this secure industrial remote operations platform—integrating Proxmox VE virtualization, Apache Guacamole remote desktop, MQTT-based IoT robot control, and AI-powered demand forecasting—has been an invaluable experience for my professional growth in full-stack development and industrial systems integration.

I extend gratitude to EPI Digital School's administration and faculty for providing quality education and academic resources essential for understanding enterprise-grade web application development, security architecture, and modern software engineering practices.

Finally, I am grateful to my family and friends for their unwavering support and encouragement throughout this journey.

Contents

List of Figures

List of Tables

General Introduction

The Fourth Industrial Revolution, or Industry 4.0, has fundamentally transformed industrial operations by integrating artificial intelligence (AI), the Internet of Things (IoT), cloud computing, and advanced automation into manufacturing and operational processes. As industrial systems become increasingly digitized and interconnected, engineers and technicians require secure, remote access to specialized equipment — programmable logic controllers (PLCs), human-machine interfaces (HMIs), SCADA systems, and industrial robots — for training, development, and maintenance purposes. Traditional approaches to industrial remote access present significant limitations: VPN-based solutions are complex to configure and, in many Operational Technology (OT) deployment contexts, incompatible with OT security requirements; conventional remote desktop tools lack native integration with industrial communication protocols; and no standardized platform has emerged to consolidate virtual machine (VM) provisioning, browser-based remote desktop access, and IoT device control within a single, coherent system. As the comparative analysis presented in Chapter 1 will demonstrate, existing solutions address these challenges only partially. This raises a central engineering question: *how can industrial engineers and trainers securely access remote equipment from any browser, without client-side software installation, while preserving OT network integrity and enforcing strict access control?*

It is within this industrial and technological context that the present internship was conducted at **Novation City** in Sousse, Tunisia, from FEB 2026 to MAY 2026, as part of the final-year project requirements for the Software Engineering Bachelor at EPI Digital School. Novation City is a competitiveness cluster established in 2009 as a public-private partnership dedicated to fostering innovation and industrial transformation within the mechatronics and Industry 4.0 sectors [1]. Novation City operates through several specialized business units, among which **Novation Akademy** (industrial training and professional skills development) and **Innovation** (applied research and R&D initiatives) are the primary drivers of the operational requirement for a secure remote laboratory infrastructure. This convergence of training needs and R&D capability motivated the development of a dedicated platform to provide trainees and engineers with secure remote access to industrial equipment without the constraints of physical presence.

To address these challenges, Novation City initiated the **IoT-REAP** (**Internet of Things — Remote Equipment Access Platform**) project. The platform provides industrial engineers, technicians, and trainers with browser-based access to virtual machines through Proxmox VE integration [2], and clientless remote desktop sessions via Apache Guacamole, supporting the RDP, VNC, and SSH protocols [3]. Robot commands are dispatched and monitored via the lightweight MQTT messaging protocol [4], enabling real-time communication with connected industrial devices. IoT-REAP adheres to strong industrial cybersecurity principles,

implementing zero-trust access controls and enforcing OT network segmentation drawing upon the principles of the Purdue reference model. The platform requires no client-side software installation, thereby reducing the client-side attack surface and simplifying access provisioning for geographically distributed users.

The primary objective of this internship was therefore to design and implement the IoT-REAP platform, with specific focus on the development of the unified web portal, the integration of Proxmox VE and Apache Guacamole for virtualization and remote desktop orchestration, and the implementation of the secure MQTT-based IoT control interface. The deployment of underlying network infrastructure, the administration of the Proxmox hypervisor layer, and the development of proprietary hardware drivers lie outside the scope of this report and were addressed by Novation City's internal IT operations team as part of the broader infrastructure initiative.

The report comprises three chapters documenting the complete development lifecycle of IoT-REAP. **Chapter 1** establishes the professional context at Novation City, characterizes the challenges of secure industrial remote access, critically reviews existing industrial remote access solutions and their limitations, defines the platform requirements and scope, and presents the Agile Scrum methodology adopted, including the six-sprint development timeline. **Chapter 2** presents the conceptual study of the platform, describing the adopted MVC software architecture extended with Service and Repository layers, justifying the technology stack selection, and modeling the system through a general use case diagram and actor-specific refinements, a class diagram formalizing the domain model, and sequence diagrams capturing the dynamic behavior of critical workflows including VM session provisioning, robot command dispatch, and user authentication. **Chapter 3** covers the realization and experimentation phase, presenting the hardware and software development environment, the principal interfaces implemented for each user role, the network topology and segmentation strategy, the production deployment and security configuration, and the verification activities confirming functional correctness, role-based access control, remote laboratory workflows, and performance. The report closes with a critical assessment of achievements and limitations encountered during development, and concrete recommendations for future platform enhancements.

Chapter 1

Preliminary Study

Introduction

This chapter presents the preliminary study of the IoT-REAP project. It situates the work within the Novation City organizational context, establishes the problem statement motivating the platform, reviews existing industrial remote access solutions and their limitations, defines the proposed platform and its requirements, and concludes with the Agile Scrum methodology and sprint planning that structured the implementation.

1 General Project Context

This section situates the IoT-REAP project within its professional and academic framework. It describes the host organization, its business units, infrastructure strategy, and the academic scope of the internship, all of which explain the context in which a secure remote industrial access platform was identified as a necessary development.

1.1 Host Organization

Novation City is a competitiveness cluster established in 2009 in Sousse, Tunisia, as a public-private partnership. The cluster is designed to foster innovation ecosystems and enhance industrial competitiveness through digital transformation and advanced technologies, addressing the growing demand for mechatronics expertise and Industry 4.0 adoption in the MENA region [1].

Figure 1.1: Novation City Official Logo and Branding

Mission Statement: Novation City's mission is to foster an innovation ecosystem in the mechatronics sector by bringing together companies, industrial entities, research centers, universities, and engineering schools. The cluster aims to enhance regional and national competitiveness through technology advancement, knowledge transfer, and strategic investment attraction.

Vision: The cluster envisions becoming a leading competitiveness hub for Industry 4.0 adoption and digital transformation in North Africa and the broader MENA region, focusing on

advanced innovation infrastructure, highly skilled technical talent development, and sustainable academia-industry partnerships.

1.2 Business Units and Services

Novation City operates through four specialized business units, each addressing a distinct aspect of the innovation ecosystem.

Acceleration Unit: Supports startup growth through business incubation services, mentorship programs, and access to funding opportunities. Services include workspace provision, networking facilitation, and investor connections.

Advisory Unit: Delivers strategic consulting focused on digital transformation and Industry 4.0 implementation. Advisory services encompass digital readiness assessments, technology selection, change management, security compliance audits, and infrastructure planning for remote operations platforms.

Formation Unit (Novation Akademy): Provides training and skills development programs covering mechatronics, automation, digital technologies, project management, and innovation methodologies in collaboration with universities and industry partners.

Innovation Unit: Drives research and development initiatives, facilitating collaborative R&D between academic institutions and industrial partners while exploring emerging technologies for the regional ecosystem.

1.3 Infrastructure and Strategic Focus

Novation City manages three specialized geographic zones:

- **Mechatronic City:** Houses companies focused on mechatronics, robotics, and advanced manufacturing, supported by specialized laboratories and testing facilities.
- **Business City:** Accommodates service-oriented companies and consulting firms with modern office spaces and collaborative environments.
- **Industrial City:** Dedicated to manufacturing operations with infrastructure supporting advanced manufacturing processes and Industry 4.0 technologies.

A central pillar of Novation City's strategy is promoting Industry 4.0 principles and digital transformation. As documented in the industrial adoption literature [1], many manufacturing companies face significant challenges in implementing Industry 4.0 technologies, including limited awareness, uncertainty about return on investment, skills gaps, and integration

difficulties with legacy systems. To address these challenges at scale, Novation City extends its capabilities with secure remote access platforms that enable engineers to interact with industrial systems remotely through the IoT-REAP initiative, combining Infrastructure-as-a-Service provisioning with zero-trust security principles.

1.4 Academic Framework

This project was carried out as part of a licence-level software engineering internship at Novation City. The academic objective was to apply web engineering, software architecture, database modeling, remote infrastructure integration, and secure system design to a concrete industrial problem. The work required formal documentation through UML modeling, requirements analysis, implementation reporting, and validation of the delivered platform against specified requirements. The project thus bridges academic software engineering practice with real Industry 4.0 operational needs.

2 Problem Statement

The IoT-REAP project addresses critical operational challenges facing industrial companies and training institutions within the Industry 4.0 ecosystem. As manufacturing systems become increasingly digitized and interconnected, engineers and technicians require hands-on access to specialized industrial equipment—PLCs (Programmable Logic Controllers), HMIs (Human-Machine Interfaces), SCADA systems, and industrial robots—for training, development, and maintenance purposes.

Remote Access Limitations: Traditional approaches to industrial system access present significant challenges:

- Physical presence requirements limit training scalability and increase travel costs
- VPN-based solutions are complex to configure, difficult to maintain, and often incompatible with OT (Operational Technology) security requirements
- Existing remote desktop tools lack integration with industrial protocols and equipment
- No unified platform exists for VM provisioning, remote desktop access, and IoT device control

Security Concerns in OT Environments: Industrial networks require strict isolation following the Purdue Reference Model (which organizes industrial network zones into five hierarchical levels, from field devices at Level 0 through control systems at Levels 2–3 to enterprise IT at Level 4–5), yet traditional IT remote access tools are not designed for OT security requirements:

- **Network Segmentation:** IT remote access solutions often bypass OT network boundaries, creating security vulnerabilities
- **Session Management:** No automatic session expiration or quota management for shared industrial resources
- **Audit Trail:** Insufficient logging for industrial security standards
- **Access Control:** Lack of fine-grained role-based access to different industrial zones

Resource Sharing Challenges: Educational institutions and industrial companies face difficulties in sharing expensive equipment:

- **Equipment Scarcity:** Industrial robots and specialized virtual machines are costly and limited in quantity
- **Scheduling Conflicts:** No automated reservation system prevents double-booking of shared resources
- **Utilization Monitoring:** Inability to track usage patterns and optimize resource allocation

Table 1.1: Critical Limitations of Current Industrial Remote Access Solutions

Limitation Category	**Specific Problem**	**Business Impact**
Physical Access	Engineers must be on-site to access equipment	Limited training capacity, high travel costs
VPN Complexity	Complex configuration, security vulnerabilities	IT overhead, OT security risks
Fragmented Tools	Separate systems for VMs, remote desktop, robots	Context-switching, integration difficulties
No Session Management	No automatic expiration or resource quotas	Resource conflicts, security exposure
Limited Audit Trail	Insufficient logging for compliance	Industrial security gap

3 Study of the Existing

The limitations established in Section 2 motivate a systematic examination of existing industrial remote access solutions. Before proposing the IoT-REAP architecture, it is essential to

understand the technical approaches currently in use and identify the specific gaps they fail to address. This analysis provides the foundation for the design decisions presented in Section 4.

3.1 Analysis of Existing Industrial Remote Access Solutions

Industrial remote access has traditionally relied on several established approaches, each with distinct characteristics and limitations.

Virtual Private Networks (VPNs): The most common approach for industrial remote access involves establishing encrypted tunnels between remote users and industrial networks. Organizations deploy VPN concentrators at facility perimeters, requiring engineers to install client software, configure connections, and authenticate before accessing internal resources.

Jump Servers / Bastion Hosts: Some organizations implement intermediate servers that act as controlled access points to OT networks. Engineers first connect to the jump server, then establish secondary connections to target systems. This adds a security layer but increases connection complexity and latency.

Proprietary Remote Access Tools: Industrial equipment vendors (Siemens, Rockwell, Schneider) offer vendor-specific remote access solutions tied to their hardware ecosystems. These tools provide optimized connections to specific PLCs and HMIs but create vendor lock-in and fragmentation across multi-vendor environments.

Cloud-Based IIoT Platforms: Emerging platforms such as AWS IoT, Azure IoT Hub, and PTC ThingWorx provide cloud connectivity for industrial devices. However, these focus primarily on data collection and monitoring rather than interactive remote control and desktop access.

Generic Remote Desktop Solutions: Tools such as TeamViewer, AnyDesk, and Microsoft RDP provide remote desktop capabilities but lack industrial-specific features: OT network isolation, session quotas, audit logging for compliance, and integration with virtualization platforms.

Recent academic literature has complemented these commercial developments with proposals for software-defined perimeters (SDP) in operational technology networks and blockchain-based tamper-proof audit frameworks for industrial IoT. While these contributions advance specific security or transparency properties, they typically address isolated aspects—network isolation or access control—rather than providing an integrated platform that unifies VM provisioning, browser-based remote desktop, IoT device control, and resource scheduling within a single governed framework. IoT-REAP therefore positions itself as an integrated engineering response to this fragmented academic and commercial landscape.

3.2 Criticism of the Existing

The current industrial remote access landscape exhibits several fundamental deficiencies that prevent it from meeting Industry 4.0 operational requirements.

VPN Complexity and Security Risks: VPN deployments require significant IT expertise for configuration and maintenance. Split tunneling creates security vulnerabilities, and VPN credentials are prime targets for attackers. Once inside the VPN, lateral movement across the network is often unrestricted, violating zero-trust principles.

Lack of Session Management: Existing solutions provide no automated session expiration, resource quotas, or reservation systems. Shared equipment cannot be efficiently scheduled, leading to conflicts and underutilization.

No OT/IT Network Isolation: Traditional remote access tools do not enforce Purdue model network segmentation. Engineers accessing OT systems from IT networks introduce security vulnerabilities that weaken industrial security controls.

Missing Audit Trail: Industrial security practices require comprehensive logging of all access and commands. Existing tools lack tamper-evident audit logs with cryptographic integrity verification.

Fragmented Tooling: Organizations must deploy separate systems for VM management, remote desktop, IoT device control, and camera monitoring. This fragmentation increases operational complexity and prevents unified security policies.

Table 1.2: Comparison of Existing Industrial Remote Access Solutions

Requirement	**VPN Solutions**	**Generic RDP Tools**	**IIoT Platforms**
Browser-based Access	Poor: Client installation required	Poor: Client software required	Good: Native web interface
VM Provisioning	Poor: Not supported	Poor: Not supported	Poor: Data focus only
Session Management	Poor: Manual only	Poor: No automation	Partial support
OT/IT Isolation	Poor: Bypasses zones	Poor: No awareness	Partial support
Audit Logging	Good: Basic logs	Poor: Insufficient	Good: Comprehensive logging
IoT Integration	Poor: Not supported	Poor: Not supported	Good: Core feature

The systematic analysis reveals that current solutions were designed for traditional IT

environments and fail to address the unique requirements of Industry 4.0: zero-trust security, OT compliance, resource-efficient scheduling, and unified management. These deficiencies collectively justify the development of a purpose-built industrial remote access platform.

4 Proposed Solution

Based on the deficiencies identified in Section 3, this section presents the IoT-REAP platform definition, its architectural approach, the technologies constituting the solution, and an overview of the security framework and resource management capabilities.

4.1 Project Definition

The IoT-REAP (Internet of Things – Remote Equipment Access Platform) project aims to develop an integrated web-based platform that provides secure remote access to industrial systems, virtual machines, and IoT devices. The project addresses the limitations identified in current industrial remote access solutions by creating a unified platform for Industry 4.0 operations.

Project Scope: Development of a comprehensive platform enabling engineers to provision virtual machines, access them via browser-based remote desktop, control IoT robots, monitor camera feeds, and operate within a zero-trust security framework—all from a single unified interface, without client-side software installation.

Core Problem Addressed: Existing remote access solutions are fragmented, lack OT-specific security controls, provide no resource management, and fail to meet industrial compliance requirements. IoT-REAP unifies VM provisioning, remote desktop access, IoT device control, and security monitoring into a single governed platform.

4.2 Solution Architecture

The proposed solution employs a modern web application architecture built on established software engineering principles.

Separation of Concerns: Clear architectural boundaries between data management, business logic, and presentation layers enable independent development and maintenance of each component. The backend follows the Service-Repository pattern, ensuring controllers remain thin and business logic is encapsulated in dedicated service classes.

API-First Integration: External services (Proxmox VE, Apache Guacamole, and MQTT broker) integrate through well-defined REST interfaces, ensuring loose coupling and flexibility

to adapt to evolving infrastructure [2, 3, 4].

Event-Driven Architecture: System components communicate via Laravel Events and WebSocket broadcasts, enabling real-time updates for session status, robot telemetry, and security alerts [5].

Zero-Trust Security Model: Every access request is authenticated and authorized regardless of network location, following least-privilege principles with role-based policy enforcement.

4.3 Technology Overview

Table 3.3 maps each layer of the IoT-REAP platform to its corresponding technology and functional role. This overview provides a consolidated reference for understanding the subsequent implementation chapters.

Table 1.3: IoT-REAP Technology Stack

Layer	**Technology**	**Role in Platform**
Backend Framework	Laravel 12 (PHP)	REST API, business logic, event broadcasting, job queues
Frontend	React 18 + TypeScript	Reactive user interface; Zustand state management
Virtualization	Proxmox VE	VM lifecycle management and node orchestration
Remote Access Gateway	Apache Guacamole	Browser-based RDP/VNC/SSH tunneling
IoT Messaging	MQTT (Mosquitto)	Robot command publishing and telemetry streaming
Relational Database	MySQL 8	Persistent storage for all platform entities

5 Requirements Specification

The requirements specification translates the problem statement and proposed solution into concrete actors, system services, and quality constraints. It identifies who interacts with the platform, what each actor must be able to do, and the measurable quality properties expected from the solution.

5.1 Identification of Actors

The platform is organized around role-based access control implemented through three application roles: `engineer`, `teacher`, and `admin`. An unauthenticated state represents public access. Each actor interacts with the same platform from a different operational perspective, and each role is associated with a distinct set of services, permissions, and workflows.

Unauthenticated User (Guest) represents any visitor who has not yet authenticated. This actor can browse public training paths, access legal and informational pages, consult public discussion threads in read-only mode, view training path reviews, verify public certificates, initiate authentication and registration workflows, and access public checkout result pages after external payment redirection.

Engineer is the primary learner and remote laboratory user. This actor enrolls in training paths, consumes educational resources (articles, videos, quizzes), tracks progress, writes notes, participates in discussions, posts reviews, claims certificates, and initiates payment and refund operations. Beyond the learning dimension, the engineer also provisions virtual machine sessions, opens browser-based remote desktop access through Guacamole, configures connection preferences, reserves USB devices and cameras, attaches hardware to active sessions, enters waiting queues when devices are busy, and interacts with session-bound laboratory resources.

Teacher is the content producer and pedagogical supervisor. This actor creates and manages training paths, modules, and training units; authors articles; uploads and manages videos and captions; creates and publishes quizzes; reviews learner interactions through analytics dashboards; moderates instructional discussions; submits training paths for administrative approval; and requests payout for generated revenue.

Administrator is the platform governance and infrastructure actor. This role supervises users, teacher approval, training path approval, featured training path ordering, payment-related review processes, payout validation, infrastructure supervision, Proxmox server and node management, VM assignment approval, hardware gateway verification, camera and reservation management, flagged forum content moderation, maintenance operations, alerts, and activity logging. Administrators also hold compliance-sensitive permissions such as user impersonation, suspension, and deletion.

Stripe (External Payment Processor) is an external system actor that supports platform commerce workflows. IoT-REAP integrates with Stripe through hosted checkout redirections and signed webhook callbacks to confirm payments, create internal payment records, and react to asynchronous state changes. Stripe also participates in finance workflows such as refunds and instructor payouts [7].

Google OAuth (External Identity Provider) is an external system actor that provides social authentication services. The platform delegates identity verification to Google's OAuth 2.0 endpoint, allowing users to authenticate using existing Google credentials. Upon successful authorization, Google returns verified identity claims that the platform uses to create or link local user accounts, reducing registration friction.

Camera Hardware (Secondary Actor) represents the ESP32-CAM and compatible PTZ units deployed in the laboratory environment. These devices stream MJPEG video feeds to the platform gateway and accept pan-tilt-zoom control commands initiated by authorized users. The platform mediates all camera interactions, handling stream proxying, resolution negotiation, and reservation state synchronization.

The interaction among these actors reveals that the platform combines four complementary dimensions: e-learning delivery, instructor authoring, virtual laboratory provisioning, and infrastructure governance. This combination distinguishes IoT-REAP from a conventional learning management system because educational workflows are directly coupled to remote computing resources and physical IoT device access.

5.2 Functional Requirements

The platform must support the following functional requirements according to actor responsibilities and shared system behavior.

Unauthenticated User (Guest):

- Access account registration and login workflows
- Consult the landing page and public legal pages
- Browse public training paths and training path details
- Read public reviews and forum threads in read-only mode
- Verify and download publicly accessible certificates through verification links
- View public search results, trending queries, and category-based discovery
- Access checkout success and cancellation pages after external payment flows

Engineer:

- Enroll in and unenroll from training paths
- Access owned or enrolled training units and learning materials

- Read articles and track article completion
- Stream instructional videos and update playback progress
- Attempt quizzes, submit answers, and review attempt history
- Mark training units complete or incomplete based on learning progress
- Create, update, and delete personal notes for training units
- Create, edit, and delete reviews for completed or relevant training paths
- Participate in forum discussions by creating threads, replying, voting, and flagging content
- View notifications and manage read status
- Claim, verify, and download certificates
- Initiate checkout for paid training paths
- Review payment history and submit refund requests
- Provision, extend, view, and terminate virtual machine sessions
- Retrieve Guacamole access tokens for remote desktop connectivity
- Manage connection preference profiles for available protocols
- Browse available Proxmox virtual machines and snapshots
- Reserve USB devices and cameras
- Attach or detach session hardware and join hardware waiting queues
- Access session camera streams, request PTZ control, and adjust stream resolution

Teacher:

- Create, edit, archive, restore, and delete training paths
- Organize training paths into modules and training units
- Reorder modules and training units
- Create and maintain quiz structures, questions, and publication status
- Create, update, and delete article content

- Upload, retry, and delete training videos
- Manage video captions
- Submit training paths for administrative review
- View teacher analytics for enrollment, earnings, and learning funnel data
- Access a teacher discussion inbox and moderate course-related threads
- Request revenue payout
- Submit training unit virtual machine assignment requests

Administrator:

- Access the administrative dashboard and KPI views
- Approve, reject, feature, unfeature, and order training paths
- Review pending VM assignment requests
- Manage reservations for devices and cameras
- Manage Proxmox servers and nodes
- Start, stop, reboot, and shut down virtual machines through node operations
- Register, verify, update, and remove gateway nodes
- Discover hardware devices and refresh infrastructure status
- Bind, unbind, attach, detach, dedicate, and configure devices and cameras
- Assign and unassign cameras to virtual machines
- Approve teacher accounts and manage user roles
- Suspend, unsuspend, impersonate, and delete user accounts where authorized
- Review and process refund requests
- Review, approve, reject, and process payout requests
- Monitor video processing status
- Place devices and cameras in maintenance mode
- Moderate flagged threads and replies

- Manage system alerts and consult activity logs

System-Wide Functional Requirements:

- Enforce authentication and email verification for protected features
- Apply authorization policies according to role and ability
- Persist learning, payment, infrastructure, and moderation data
- Support public and protected routes through a unified web interface
- Provide rate limiting for sensitive or abuse-prone operations
- Maintain notification and audit mechanisms for critical events
- Support asynchronous or stateful workflows (video processing, VM lifecycle operations, refund handling, hardware attachment)

5.3 Non-Functional Requirements

The quality constraints of the platform are defined by its educational, infrastructure, and operational nature. Where applicable, measurable thresholds are specified to allow objective verification during the testing phase.

Performance:

- API response time for standard CRUD operations must not exceed 500 ms under normal load conditions
- VM provisioning and session initialization must complete within 30 seconds for standard configurations
- Remote desktop connection establishment through Guacamole must not exceed 5 seconds post-authentication
- Video streaming and progress tracking must occur with perceptible latency under 2 seconds
- Search, analytics summaries, and notification retrieval must execute within acceptable response windows under concurrent demand

Security:

- Authentication must rely on secure identity management with mandatory email verification
- Authorization must be enforced by middleware and policy gates at every protected endpoint
- Payment workflows must validate external webhook signatures and preserve transaction integrity
- Access to VM sessions, cameras, and USB devices must be restricted to authorized users with active reservations
- Sensitive administrative actions must generate tamper-proof audit records

Reliability:

- Learning progress, notes, payments, and reservations must remain durable and consistent under concurrent access
- Hardware and infrastructure actions must tolerate transient failures and preserve observable system state
- Session resources must support full lifecycle management including creation, extension, hibernation, and cleanup
- Alerting and logging mechanisms must assist recovery from operational issues

Usability:

- The user interface must remain coherent and navigable for learners, teachers, and administrators without requiring specialized infrastructure knowledge
- Administrative screens must expose complex operations in a controlled and structured form
- Forum, notes, review, and certificate features must integrate naturally into the learning workflow

Scalability and Maintainability:

- The architecture must support extension through modular routes, controllers, models, and services without disrupting existing functionality
- Domain separation between learning, commerce, virtual laboratory, and infrastructure operations must remain clearly bounded

- The platform must support future growth in training content, users, devices, and compute nodes without architectural refactoring
- The codebase must be supported by automated tests, strongly typed frontend code, and structured backend services

6 Work Methodology

The development of IoT-REAP required a structured approach to coordinate complex technical requirements with evolving stakeholder needs. This section presents the Agile Scrum methodology selected for the project, the product backlog that governed sprint priorities, the associated ceremonies, and the UML modeling language used for system documentation.

6.1 The Agile Scrum Methodology

Agile Scrum was selected for IoT-REAP because the platform combines evolving infrastructure requirements (Proxmox integration, Guacamole, MQTT), camera and robot integration, and iterative stakeholder validation—conditions that make fixed, waterfall-style planning impractical. This subsection defines the Scrum framework and describes its application to the project.

6.1.1 Definition

Agile Scrum is a structured framework for implementing Agile principles in software development [14]. While Agile is a broad philosophy emphasizing iterative delivery and adaptability, Scrum is a concrete instantiation that defines roles, ceremonies, and artifacts. Scrum organizes work into fixed-duration iterations called **sprints** (typically 1–2 weeks), during which a cross-functional team commits to delivering a potentially shippable product increment. Each sprint follows a cycle of planning, daily standups, development, review, and retrospective ceremonies.

Scrum defines three roles: the **Product Owner** (prioritizes the backlog and validates requirements), the **Scrum Master** (facilitates ceremonies and removes blockers), and the **Development Team** (delivers the increment). The framework manages work through three artifacts: the **Product Backlog** (prioritized feature list), the **Sprint Backlog** (items committed for the current sprint), and the **Increment** (working software produced at sprint end).

The Agile Scrum framework builds on the Agile Manifesto (2001) [14], which prioritizes:

- Individuals and interactions over processes and tools

- Working software over comprehensive documentation
- Customer collaboration over contract negotiation
- Responding to change over following a plan

For IoT-REAP, these principles translate directly into practice: infrastructure integration decisions (Guacamole configuration, Proxmox API behavior) were refined iteratively based on empirical results rather than upfront specification, and the training platform scope was introduced mid-project in response to stakeholder feedback without disrupting the core laboratory platform delivery.

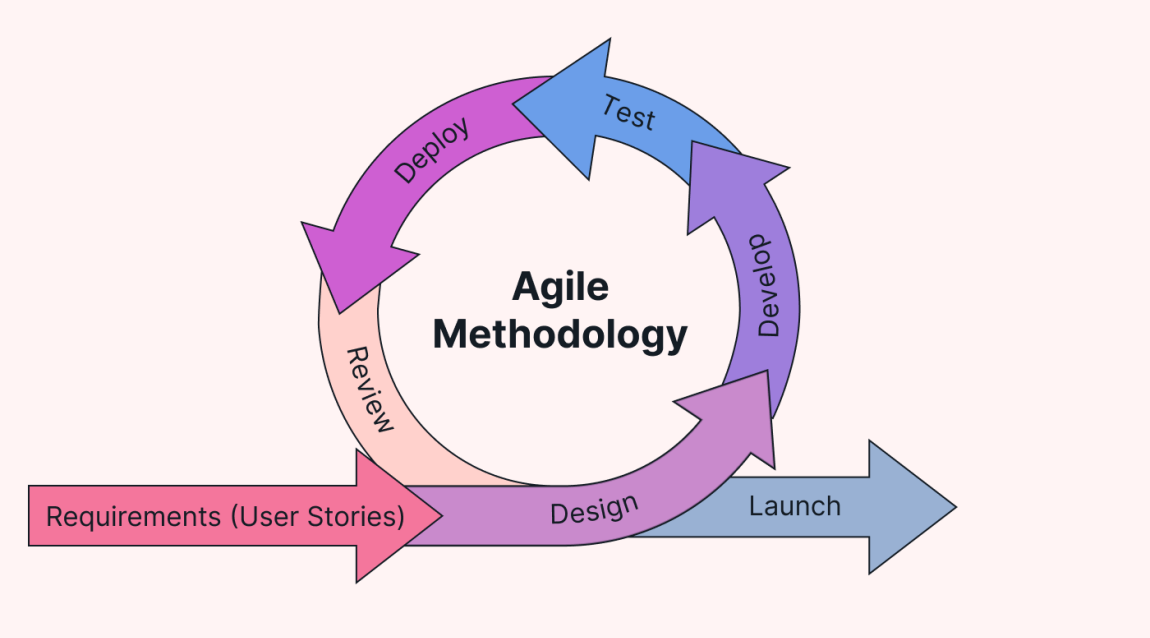


Figure 1.2: Agile Scrum Methodology Workflow

6.1.2 Characteristics of Agile Scrum

Sprint-Based Delivery: The IoT-REAP project is organized into fixed 2-week sprints, each with a starting Sprint Planning session, daily progress tracking, and a closing Sprint Review and Retrospective. This structure provides predictable delivery cycles and regular stakeholder engagement.

Sprint Planning & Commitment: At the beginning of each sprint, the team commits to specific user stories with clear acceptance criteria. In the solo developer context, this involved supervisor and Advisory Unit stakeholder alignment on sprint goals, ensuring shared understanding before development began.

Product Backlog Management: Features and requirements are maintained in a prioritized Product Backlog. Items are ordered by business value, risk, and dependency. New requirements

(infrastructure changes, the training platform addition) are incorporated into subsequent sprints without disrupting committed work.

Sprint Increments and MVP Validation: Each sprint produces a working software increment. Sprint 3 delivers the MVP (VM provisioning + remote desktop access + basic robot control), providing an early validation milestone with stakeholders before the platform's full feature set is implemented.

Sprint Review & Stakeholder Feedback: At each sprint boundary, completed work is demonstrated to stakeholders. Reviewers examine running features, validate acceptance criteria, and provide feedback that influences subsequent sprint prioritization.

Sprint Retrospective & Continuous Improvement: Following each review, retrospectives identify improvements in process, tooling, and code quality. Early retrospectives surfaced the need for more comprehensive API documentation and identified infrastructure bottlenecks requiring architectural adjustments.

Definition of Done: For IoT-REAP, a user story is considered complete when: code is written and passes automated tests, the feature is validated against its acceptance criteria, and any relevant API or user-facing behavior is documented.

6.2 Product Backlog

The IoT-REAP platform development followed a prioritized product backlog organized by sprint. Table 1.4 presents the key user stories with priorities and sprint assignments.

Table 1.4: IoT-REAP Platform Product Backlog

ID	User Story / Feature	Priority	Sprint
Sprint 1: Foundation, Authentication & Audit			
01	As a user, I want to register and log in securely using session-based authentication	High	Sprint 1
02	As an admin, I want role-based access control (engineer, teacher, admin)	High	Sprint 1
03	As a developer, I need a database schema for users with ULID primary keys	High	Sprint 1
04	As a user, I want password reset functionality via email	Medium	Sprint 1
05	As a developer, I need a React frontend with TypeScript and Zustand state management [8, 9, 10]	High	Sprint 1
06	As a developer, I need a tamper-proof audit log with SHA-256 hash chaining	High	Sprint 1
Sprint 2: Proxmox Integration, Multi-Cluster & Network Isolation			

Table 1.4 (continued)

ID	User Story / Feature	Priority	Sprint
07	As a developer, I need a database schema for VM nodes, templates, and sessions	High	Sprint 2
08	As a developer, I need a Proxmox API client with retry logic and error handling	High	Sprint 2
09	As an engineer, I want to browse VM templates and launch sessions	High	Sprint 2
10	As a developer, I need a load balancer to distribute VMs across Proxmox nodes	High	Sprint 2
11	As an admin, I want to view all nodes with live CPU/RAM statistics	Medium	Sprint 2
12	As an admin, I want to register multiple Proxmox servers with encrypted credentials	High	Sprint 2
13	As a developer, I need to extend the Proxmox architecture to support multiple independent clusters	High	Sprint 2
14	As an engineer, I want to select a target Proxmox cluster when provisioning a VM	High	Sprint 2
15	As an admin, I want to enforce Purdue model network isolation (IT/OT zone separation)	High	Sprint 2
16	As an admin, I want a complete dashboard with quota management and activity logs	High	Sprint 2
Sprint 3: Guacamole Remote Access — MVP			
17	As a developer, I need a Guacamole client service for connection management	High	Sprint 3
18	As an engineer, I want browser-based RDP/VNC access to my VM session	High	Sprint 3
19	As an engineer, I want to extend my session duration in 30-minute increments	High	Sprint 3
20	As an engineer, I want to terminate my session and have the VM cleaned up	High	Sprint 3
21	As an engineer, I want to save my preferred connection settings (display, audio)	Medium	Sprint 3
Sprint 4: Robot, Camera, Session Management & Integration Hardening			
22	As a developer, I need an MQTT service for robot command publishing	High	Sprint 4
23	As an engineer, I want to reserve a robot with atomic transaction locks	High	Sprint 4

Table 1.4 (continued)

ID	User Story / Feature	Priority	Sprint
24	As an engineer, I want an emergency stop button that terminates robot motion instantly	High	Sprint 4
25	As an engineer, I want real-time robot telemetry streamed via WebSocket	High	Sprint 4
26	As a developer, I need idle session hibernation (suspend after 10 minutes of inactivity)	Medium	Sprint 4
27	As an engineer, I want a live camera feed from the ESP32-CAM during robot control	High	Sprint 4
Sprint 5: Technical Training Platform			
41	As an instructor, I want to create structured courses with modules, lessons, and prerequisite tracking	High	Sprint 5
42	As an instructor, I want to upload videos with HLS streaming and multi-quality playback (360p–1080p) [11]	High	Sprint 5
43	As a learner, I want to enroll in courses and track my progress across lessons and modules	High	Sprint 5
44	As an instructor, I want to create timed quizzes with auto-grading, passing scores, and retry limits	High	Sprint 5
45	As a learner, I want hands-on VM lab sessions integrated with course lessons for practical training	High	Sprint 5
46	As a learner, I want to receive verifiable digital certificates upon course completion	High	Sprint 5
47	As a learner, I want to take timestamped notes during video lessons for later review	Medium	Sprint 5
48	As a learner, I want to rate and review courses to help other learners make informed decisions	Medium	Sprint 5
49	As an instructor, I want an analytics dashboard showing enrollments, completions, and revenue	High	Sprint 5
50	As an admin, I want to manage course categories and featured course placement	Medium	Sprint 5
Sprint 6: Testing & Delivery			
51	As a developer, I need to perform tests with mocked external APIs	High	Sprint 6
52	As a developer, I need load testing: 50 concurrent logins, 25 VMs, 40 Guacamole sessions	High	Sprint 6
53	As a developer, I need an OWASP ZAP scan with zero critical or high findings [27]	High	Sprint 6

Table 1.4 (continued)

ID	User Story / Feature	Priority	Sprint
54	As a user, I want comprehensive documentation (user manual, API reference)	High	Sprint 6
55	As a developer, I need to prepare and deliver a 15-minute live defense demonstration	High	Sprint 6

6.3 Agile Scrum Ceremonies and Collaboration

Sprint Planning (2–4 hours): At the beginning of each 2-week sprint, planning sessions engage the developer, supervisors, and Advisory Unit stakeholders. Sessions define sprint goals, select user stories from the prioritized Product Backlog based on business value and dependencies, and break them into implementation tasks with effort estimates using story points. The output is a Sprint Backlog of committed work with clear acceptance criteria.

Daily Standup (15 minutes): Adapted to weekly formal checkpoints (Mondays) with supervisors, the standup ceremony addresses three questions: what was completed, what will be done, and what obstacles exist. This maintains transparency and enables rapid unblocking in the solo developer context.

Sprint Review & Demo (1–2 hours): At each sprint's end, completed work is demonstrated to stakeholders, who examine running features, validate acceptance criteria, and provide prioritization feedback for subsequent sprints. The Sprint 3 review constitutes the formal MVP demonstration, confirming core platform viability before proceeding to robot control, camera integration, and compliance features.

Sprint Retrospective (1 hour): Following the review, the team examines what worked well, what can improve, and what specific action items to carry into the next sprint. Early retrospectives surfaced the need for more comprehensive API documentation and identified infrastructure bottlenecks requiring architectural adjustments.

Product Backlog Refinement (Ongoing): Between sprints, the Product Backlog is continuously refined. New requirements are added, estimated, and prioritized, ensuring Sprint Planning sessions remain efficient and focused on high-value delivery.

6.4 The UML Modeling Language

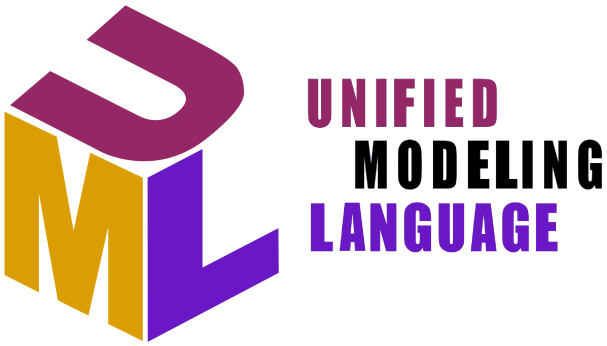


Figure 1.3: UML Modeling Language

Unified Modeling Language (UML) is a standardized visual modeling language used in software engineering to specify, visualize, construct, and document the artifacts of software systems [13]. Developed by Grady Booch, James Rumbaugh, and Ivar Jacobson in the mid-1990s, UML provides a common vocabulary for object-oriented design and is the industry standard for software architecture documentation.

UML encompasses two main categories of diagram types:

- **Structural Diagrams:** Class diagrams, component diagrams, deployment diagrams, and package diagrams
- **Behavioral Diagrams:** Use case diagrams, sequence diagrams, activity diagrams, and state machine diagrams

For the IoT-REAP platform, UML modeling was applied during the design phase to produce use case diagrams, class diagrams, and sequence diagrams illustrating critical workflows such as VM provisioning and robot command dispatch. These models served as architectural contracts between the developer and stakeholders, ensuring shared understanding of system boundaries before implementation.

7 Project Planning

The IoT-REAP platform development is organized as a phased timeline from February to end of May 2026, following an Agile Scrum sprint-based approach [14]. The project reaches 2 key milestones at Sprint 3 (MVP Demo) and Sprint 6 (Complete).

7.1 Sprint-Based Development Phases

The development timeline is organized into six thematic sprints. The chronological order reflects a deliberate architectural dependency chain: authentication and audit foundations precede infrastructure integration, which precedes remote access validation, which precedes IoT and camera integration, which precedes the training platform, and finally testing and delivery. Durations were allocated based on implementation complexity, external API stability, and the need for stakeholder demonstration buffers.

Sprint 1: Foundation, Authentication & Audit (2 weeks)

This phase established the technical baseline and security audit foundation. Laravel 12 and React 18 were integrated via Breeze to provide a full-stack scaffold. Session-based authentication with role-based access control (engineer, teacher, admin) was implemented, alongside the user database schema using ULID primary keys. Google OAuth social login was integrated as an alternative authentication path, with account linking logic to merge social and local credentials. Concurrently, a tamper-evident audit log mechanism using SHA-256 hash chaining was implemented to support cryptographic integrity of audit trails. The MySQL development environment was configured to ensure reproducible builds across development and deployment contexts.

Sprint 2: Proxmox Integration, Multi-Cluster & Network Isolation (4 weeks)

The virtual laboratory backbone was built during this phase. A Proxmox VE API client was developed with retry logic and structured error handling to tolerate transient node failures. The database schema for VM nodes, templates, and sessions was finalized. An engineer-facing VM template catalog with session launch capability was implemented, alongside an admin dashboard displaying live CPU and RAM statistics per node. A load balancer distributing VM provisioning requests across available nodes was completed. The architecture was extended to support multiple independent Proxmox clusters with encrypted credential storage and cluster selection logic for VM launch. Purdue model network isolation was enforced through VLAN and firewall rule configuration separating IT and OT zones. The administrative dashboard was extended with quota management, VM template catalog governance, and centralized activity logging.

Sprint 3: Guacamole Remote Access — MVP Demo (2 weeks)

Apache Guacamole was deployed and integrated with the Laravel backend via its REST API. The implementation covered connection creation, one-time token generation for secure URL distribution, and an embedded React viewer for browser-based RDP and VNC access. Session lifecycle management—extension in configurable increments, explicit termination with automatic VM cleanup, and persistent connection preferences—was delivered. The three-week duration accounts for the complexity of WebSocket-based display protocol tunneling, browser

compatibility testing, and the need for a stable MVP demonstration. **This phase concluded with the MVP demonstration, validating the core value proposition of browser-based remote industrial access.**

Sprint 4: Robot, Camera, Session Management & Integration Hardening (3 weeks)

The IoT control layer was built on MQTT (Mosquitto) with QoS 1 delivery guarantees for robot command publishing. An atomic reservation system using database transaction locks prevented double-booking of the physical robot unit. Real-time telemetry streaming was implemented via WebSocket for live sensor feedback, with hardened reconnection logic to tolerate network fluctuation. An emergency stop mechanism with immediate command termination was integrated as a safety-critical feature. The ESP32-CAM hardware layer was concurrently integrated to provide live MJPEG video feeds during robot control sessions, with PTZ command relaying and stream proxying through the platform gateway. Idle session hibernation was implemented: VMs are automatically suspended after 10 minutes of inactivity, reducing resource contention without terminating the session context. Camera reservation state was synchronized with active robot sessions to ensure coherent hardware allocation. This phase also addressed residual integration gaps discovered during concurrent usage testing: race conditions in the VM provisioning pipeline were resolved, and Google OAuth account linking edge cases were hardened.

Sprint 5: Technical Training Platform (4 weeks)

This phase introduced the e-learning dimension in response to stakeholder feedback following the MVP. The course structure—training paths, modules, and training units with prerequisite tracking—was implemented. Instructors gained the ability to upload videos processed into HLS adaptive streams (360p–1080p), create timed quizzes with auto-grading and retry limits, and monitor learner progress through an analytics dashboard. Engineers could enroll in courses, track completion, take timestamped notes, and receive verifiable digital certificates with unique hash-based verification. The four-week duration reflects the substantial scope of a full learning management system: video pipeline integration, progress tracking logic, quiz engine, certificate generation, and course review mechanics.

Sprint 6: Testing & Delivery (2 weeks)

The final phase concentrated on quality assurance and documentation. Unit and integration tests were performed with mocked external APIs (Proxmox, Guacamole, Stripe). Load testing validated 15 concurrent logins, 5 active VMs, and 10 simultaneous Guacamole sessions. An OWASP ZAP security scan was executed to identify and remediate vulnerabilities. User manuals, API reference documentation, and the defense demonstration were prepared for final delivery.

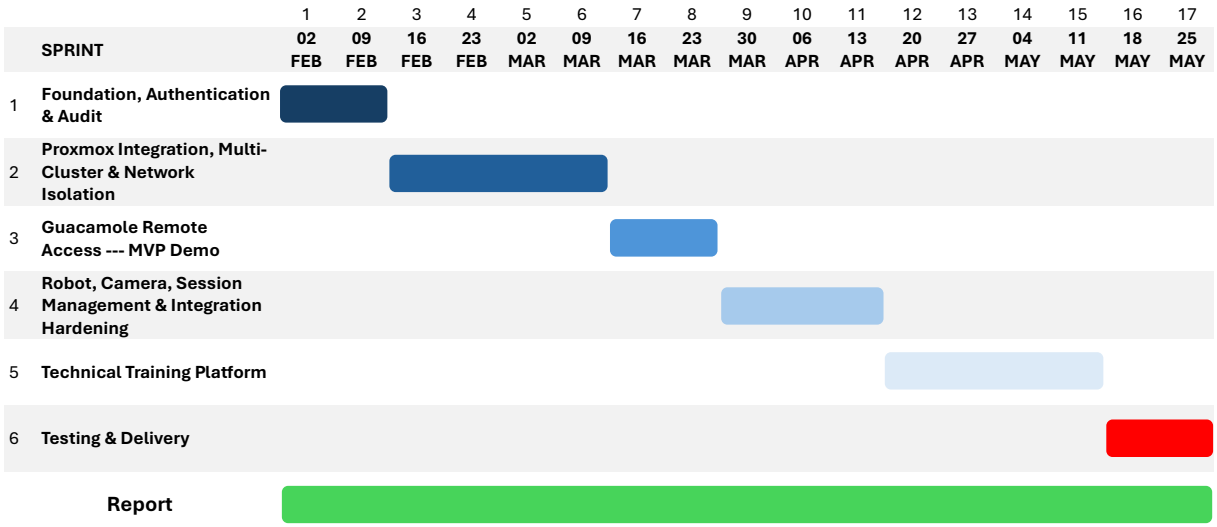


Figure 1.4: IoT-REAP Platform Development Timeline — Sprint Schedule and Key Milestones

Conclusion

This chapter has established the organizational and academic context of the IoT-REAP project, identified the operational problem motivating its development, examined the limitations of existing industrial remote access solutions, and presented the platform's proposed architecture and technology stack. The requirements specification defined the actors, functional capabilities, and measurable quality constraints governing the platform. Finally, the Agile Scrum methodology and sprint-based development timeline were introduced to frame the implementation work presented in the following chapters.

Chapter 2

Conceptual Study

Introduction

This chapter presents the conceptual study of the IoT-REAP platform. It describes the adopted software architecture, models the system through use case diagrams and representative sequence diagrams, and presents the class diagram used to guide realization.

1 Adopted Software Architecture

This section describes the architectural choices made for IoT-REAP. It first introduces the Model-View-Controller pattern in general terms, then maps each architectural layer to the concrete technologies and components of the platform. It subsequently justifies the selection of the technology stack and evaluates the benefits this structure provides with respect to the project requirements.

1.1 Architectural Pattern Overview

The Model-View-Controller (MVC) architectural pattern is a foundational design principle in software engineering that promotes separation of concerns, maintainability, and scalability [18]. Originally conceived for monolithic desktop applications, MVC structures the interaction between data, business logic, and presentation by dividing the application into three interconnected roles:

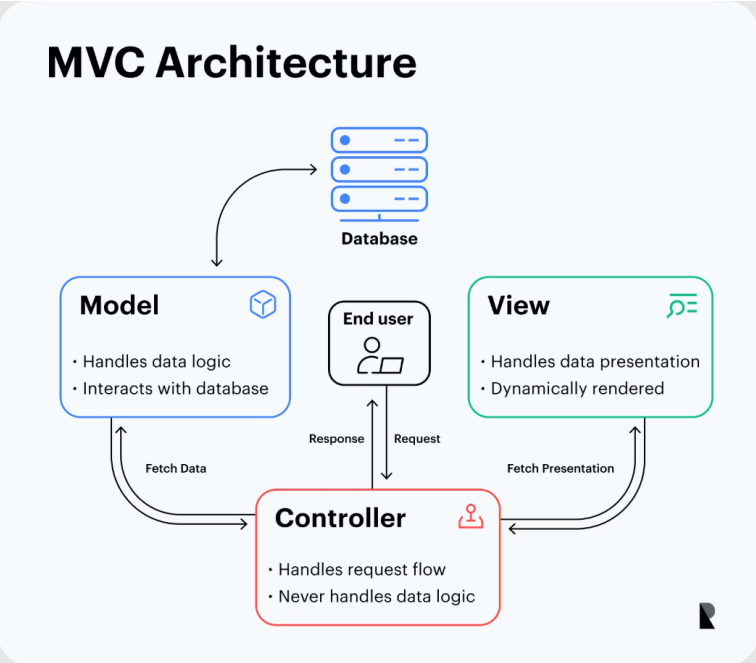


Figure 2.1: MVC Architectural Pattern: Component Interaction Flow

Model Role: Represents data structures, business logic, and data management rules. Responsible for data persistence, retrieval, validation, and enforcing business constraints independently of the user interface.

View Role: Handles the presentation layer and user interface rendering. Displays data in appropriate formats and captures user input without containing business logic.

Controller Role: Manages data flow between the Model and the View, orchestrating application logic. Receives user requests, interacts with the Model, applies business rules, and selects the appropriate View for rendering responses.

This architectural separation enables independent development, testing, and modification of each component, reducing code coupling and improving maintainability. In the context of IoT-REAP, the classic MVC pattern is extended with Service and Repository layers because the platform's complexity— decoupled React frontend, multiple external APIs (Proxmox, Guacamole, MQTT), and the need for testable persistence abstraction—requires a finer separation of concerns than the original three-role pattern provides.

1.2 MVC Layers in IoT-REAP Context

In IoT-REAP, the three conceptual MVC roles are realized through five concrete layers. The *Model* role is decomposed into three layers: **Model** (domain entities and validation), **Repository** (persistence abstraction), and **Service** (business workflow orchestration). The **Controller** role is realized by thin HTTP controllers that validate requests and dispatch to Services. The **View** role is realized entirely by the React 18 frontend.

The Model Layer defines data structures, entity relationships, and validation rules. IoT-REAP Models use Eloquent ORM to map domain entities (users, VM sessions, robots, telemetry, audit logs) to relational tables. Models enforce data integrity at the entity level but remain focused on structural definition rather than cross-entity workflow logic.

The View Layer in IoT-REAP is implemented as a React 18 frontend with TypeScript. Key principles include component-based architecture, typed props interfaces, custom hooks for data fetching, and Zustand for state management. Views render dashboards, session interfaces, robot control panels, and admin consoles [8] [9] [10].

The Controller Layer mediates between incoming HTTP requests and the Service layer, then returns JSON responses consumed by the View layer. IoT-REAP Controllers remain thin—validating input via FormRequests, delegating to Services for business logic, and returning structured responses via API Resources. Controllers enforce authentication and authorization through middleware and manage error responses uniformly.

The Service Layer orchestrates business workflows and coordinates integration with external

systems. Services such as `VMProvisioningService`, `GuacamoleClient`, and `MqttService` encapsulate use-case logic that spans multiple Models and external APIs, ensuring that Controllers remain free of complex business rules [2] [3] [4].

The Repository Layer abstracts persistence operations and query logic. Repositories such as `VMSessionRepository` and `UserRepository` encapsulate Eloquent queries, returning typed Models or Collections and shielding the Service layer from database-specific details.

Table 2.1: IoT-REAP Platform Core Controllers and Responsibilities

Controller	Primary Responsibilities
AuthController	User authentication, registration, password reset, session management
VMSessionController	Session creation, extension, termination; delegates to VMProvisioningService
VMTemplateController	Template listing, admin CRUD operations for VM templates
ProxmoxNodeController	Node stats, VM listing, VM control (start/stop/reboot)
RobotSessionController	Robot reservation, command dispatch via MqttService
SecurityEventController	Security event listing, active sessions with risk scores
AdminController	User quota management, activity logs, system configuration

1.3 Technology Stack Justification

The technology stack was selected to align each architectural layer with a stable, mature ecosystem that reduces integration risk and maximises architectural coherence. Laravel 10 provides a robust backend framework with built-in ORM, queue management, and authentication scaffolding, reducing boilerplate through its convention-over-configuration approach without sacrificing architectural rigour. MySQL serves as the relational database engine, offering proven transactional guarantees and native support within the Laravel Eloquent ORM. React 18 with TypeScript ensures type-safe component reusability across a large UI surface, while Zustand provides a lightweight, hook-based global state management solution that avoids the overhead of heavier alternatives such as Redux. Laravel Echo, backed by a self-hosted Soketi server, delivers WebSocket broadcasting without reliance on third-party cloud services, which is critical for real-time robot telemetry in constrained network environments. Proxmox VE was chosen as the open-source hypervisor to avoid proprietary licensing costs while exposing a full REST API for automated VM lifecycle management [2]. Apache Guacamole enables browser-native remote desktop access without client-side installation, which is essential for accessibility in educational contexts [3]. MQTT serves as the lightweight publish-subscribe broker for IoT telemetry, offering minimal overhead and reliable

message delivery under constrained network conditions [4].

1.4 MVC Benefits for IoT-REAP Platform

The MVC architectural pattern, extended with Service and Repository layers, provides critical advantages for a platform of IoT-REAP's complexity:

Separation of Concerns: Database specialists focus on Model optimization, frontend concentrate on React components, and backend work on Services and Controllers—enabling parallel development without conflicts.

Maintainability: Database schema changes do not require React modifications; UI redesigns do not affect business logic; and the clear organizational structure enables rapid onboarding.

Testability: Models and Services can be unit tested independently; Controllers are tested through feature tests with mocked external APIs (Proxmox, Guacamole); React components are tested with React Testing Library.

Reusability: Services are reused across multiple Controllers (e.g., `ProxmoxClient` is used by both `VMSessionController` and `ProxmoxNodeController`); React hooks are shared across pages.

Scalability: Each layer scales independently based on performance requirements; external integrations scale independently from the Laravel backend.

Table 2.2: How MVC Architecture Supports IoT-REAP Platform Requirements

Requirement	**MVC Architectural Support**
VM Provisioning	Service layer encapsulates Proxmox API; Controllers orchestrate workflow
Remote Desktop	GuacamoleClient service handles connection lifecycle; Views embed viewer
Robot Control	MqttService publishes commands; WebSocket broadcasts telemetry to Views
Zero-Trust Security	Middleware layer evaluates security posture; Controllers enforce access policies
Audit Logging	Observer pattern on Models; AuditLogger service maintains hash chain
Real-Time Updates	Events broadcast via Laravel Echo; React Views subscribe via WebSocket

Beyond functional coverage, the layered architecture directly addresses the platform's non-functional requirements. The separation of concerns and the Service layer facilitate

horizontal scaling of external integrations independently from the Laravel monolith. The Repository pattern insulates the persistence layer, allowing database migration or vendor change without frontend impact. Middleware-based security evaluation and hashed audit chains satisfy security and traceability constraints, while Laravel Echo with WebSocket support meets the real-time latency requirements for robot telemetry streaming. Together, these architectural properties establish the structural foundation upon which the concrete implementation detailed in Chapter 3 is built.

2 Use Case Diagram

After identifying actors and requirements, the conceptual analysis of the platform is refined through use case modelling. The implemented application yields one general use case view and several actor-specific refinements. The general model places the platform at the centre and connects the four user actors (Guest, Engineer, Teacher, Administrator) as well as the external Stripe actor [7] to the services they invoke: public discovery, learning progression, content authoring, infrastructure access, commerce, and administrative governance.

2.1 General Use Case Diagram

The general use case diagram groups system behaviour into four principal subsystems. In this model, the Engineer actor generalizes the Guest, retaining public consultation capabilities and adding authenticated learning and virtual laboratory operations. The Teacher actor generalizes the Engineer, extending authenticated access with content authoring and instructional management. The Administrator occupies the supervisory position by controlling both business governance and infrastructure operations. Stripe is modelled as a standalone external actor that interacts with the platform through checkout, webhook callbacks, refunds, and payout-related flows. Figure 2.2 presents the general use case diagram of the IoT-REAP platform.

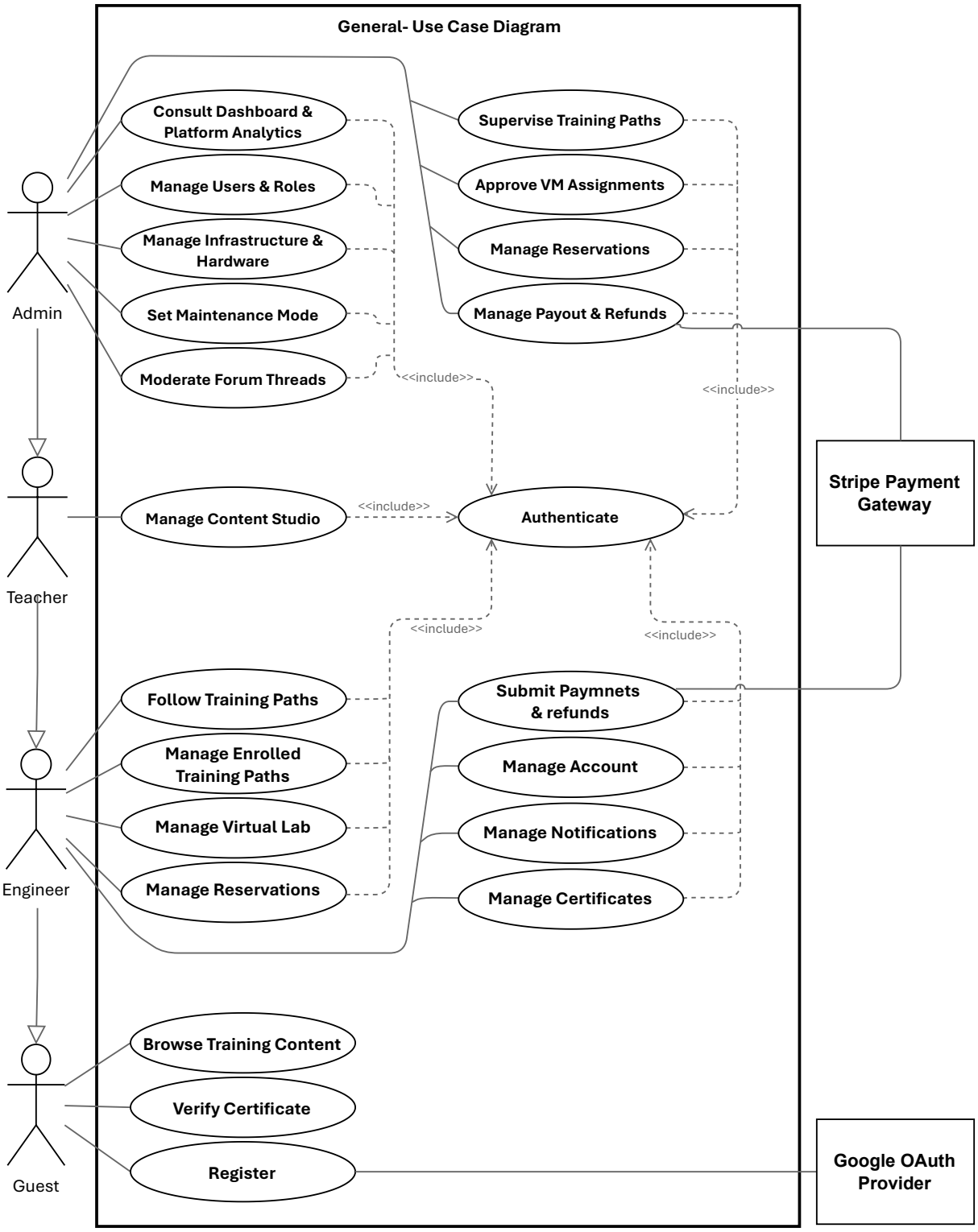


Figure 2.2: General Use Case Diagram of the IoT-REAP Platform

2.2 Actor-Specific Use Case Refinements

The following subsections present the actor-specific use case refinements derived from the implemented routes and domain behaviours of the platform. Each actor's interactions are

specified in terms of preconditions, main flows, and alternate flows, providing the traceability link between requirements and subsequent design decisions.

2.2.1 Refinement of the Use Case Manage Users & Roles

Figure 2.3 presents the refinement of the Use Case Manage Users & Roles for the Administrator actor. This use case includes the following sub-use cases:

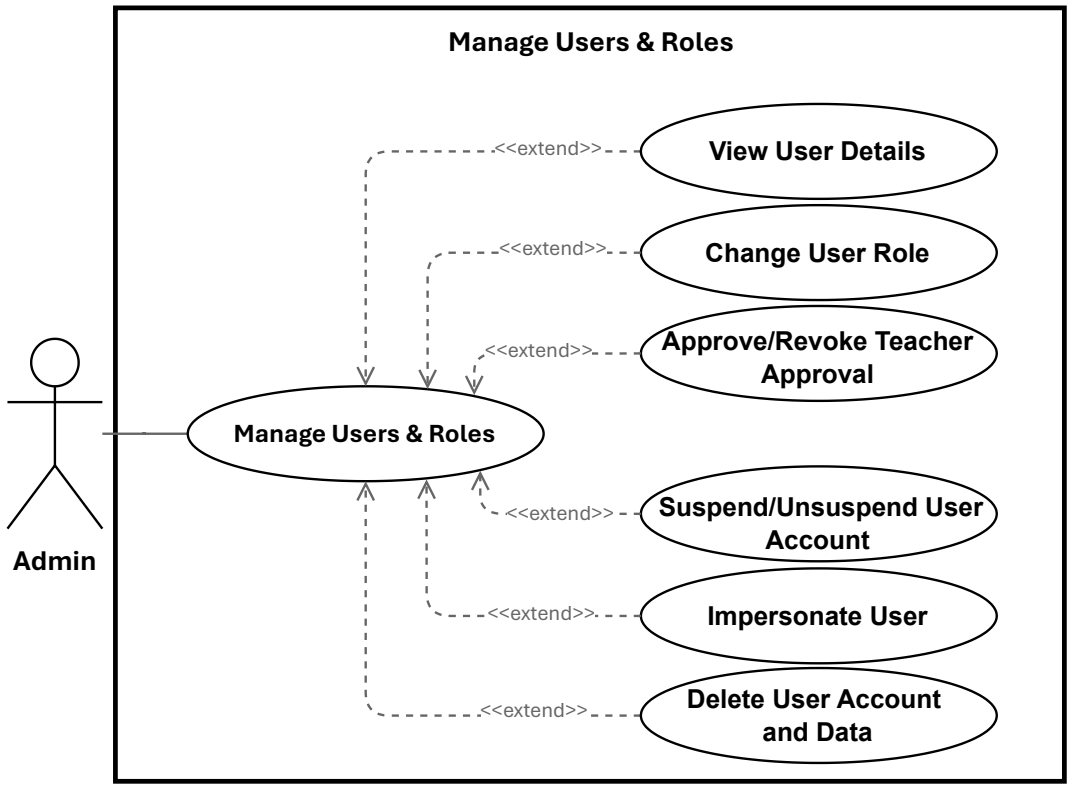


Figure 2.3: Use Case Manage Users & Roles

2.2.1.1 Specification of the Use Case Change User Role

Table 2.3: Use Case Specification: Change User Role

Title	Change User Role
Summary	Administrator updates a user's role (engineer, teacher, admin). Assigning `teacher` also records teacher-approval metadata; assigning a non-teacher clears any teacher-approval metadata.
Actor	Administrator
Precondition	Administrator is authenticated and has admin privileges. Target user exists. Requested role is one of: `engineer`, `teacher`, `admin`.
Post-condition	The user's role is persisted. If role = `teacher`, `teacher_approved_at` and `teacher_approved_by` are set; otherwise those fields are cleared. The change is logged and the updated user is returned.
Basic Scenario	1. Administrator opens the user management list or the target user's detail page. 2. Administrator selects a new role and submits the change. 3. System validates request authorization and that the role value is allowed. 4. System verifies the administrator is not attempting to change their own role. 5. System prepares the update payload (including teacher-approval metadata when applicable). 6. System persists the update to the database. 7. System logs the role change for audit purposes. 8. System returns the updated user record to the admin UI and displays confirmation.
Error Scenario	1. If the administrator attempts to change their own role, the system rejects the request and returns an error. 2. If the submitted role is invalid, the system returns a validation error and no update is made. 3. If persistence or service errors occur, the system aborts the update, logs the failure, and returns an error. 4. If the administrator is not authorized, the system returns an authorization error (403).

2.2.1.2 Specification of the Use Case Approve/Revoke Teacher Approval

Table 2.4: Use Case Specification: Approve/Revoke Teacher Approval

Title	Approve/Revoke Teacher Approval
Summary	Administrator approves a teacher account to mark it as authorized, or revokes an existing teacher approval to remove that authorization. Approving records teacher-approval metadata; revoking clears it.
Actor	Administrator
Precondition	Administrator is authenticated with admin privileges. Target user exists and has the `teacher` role.
Post-condition	If approved, `teacher_approved_at` and `teacher_approved_by` are set. If revoked, those fields are cleared. The change is logged and the updated user record is returned.
Basic Scenario	1. Administrator opens the teacher account details in the user management section. 2. Administrator selects *Approve Teacher* or *Revoke Teacher Approval*. 3. System validates that the target account is a teacher and that the request is authorized. 4. System prepares the update payload for the selected action. 5. System persists the approval change to the database. 6. System logs the action for audit purposes. 7. System returns the updated user record to the admin UI and displays confirmation.
Error Scenario	1. If the target user is not a teacher, the system returns a validation error and no change is made. 2. If the administrator is not authorized, the system returns an authorization error (403). 3. If persistence fails, the system aborts the update, logs the failure, and returns an error.

2.2.1.3 Specification of the Use Case Suspend/Unsuspend User Account

Table 2.5: Use Case Specification: Suspend / Unsuspend User Account

Title	Suspend / Unsuspend User Account
Summary	Administrator temporarily blocks or restores user access. May specify a reason and optional expiration date for temporary suspensions. All actions are recorded for audit.
Actor	Administrator
Precondition	Administrator is authenticated and authorized to manage accounts. Target user exists.
Post-condition	User access status is updated (suspended or active). Metadata (reason, expiration) is recorded. Action is logged for audit. If suspended, user can no longer log in. If unsuspended, access is restored.
Basic Scenario	1. Administrator opens user management and selects an account. 2. Administrator chooses Suspend Account or Restore Access. 3. For suspension: Administrator optionally provides a reason and expiration date. 4. System validates administrator authorization. 5. System checks that target user exists and is eligible (e.g., not an administrator). 6. System updates account status and records metadata. 7. System blocks or restores access immediately. 8. System logs the action for audit. 9. System notifies user of status change (if configured). 10. Administrator receives confirmation and updated status is displayed.
Error Scenario	1. If administrator lacks permission, request is denied and authorization error is displayed. 2. If target user does not exist, system displays error and does not proceed. 3. If target is an administrator, system prevents suspension and displays error. 4. If action cannot be completed due to system errors, system rolls back changes, preserves account state, and displays error. 5. If notifications are configured but unavailable, core action is completed, but system flags notification failure for manual follow-up.

2.2.1.4 Specification of the Use Case Impersonate User

Table 2.6: Use Case Specification: Impersonate User

Title	Impersonate User
Summary	A privileged administrator or support agent temporarily assumes a target user's identity for troubleshooting. Impersonation is time-limited, auditable, and constrained by authorization rules.
Actor	Administrator
Precondition	Initiator is authenticated and authorized to impersonate.
Post-condition	A temporary impersonation session is created. Actions performed are applied as the target user but recorded with initiator metadata. Session start and end are logged.
Basic Scenario	1. Administrator initiates impersonation via UI or API, specifying target user ID and mode (read-only/full). 2. System validates and sanitizes request parameters. 3. System verifies the initiator's permission to impersonate the target. 4. System loads the target user record and verifies eligibility (e.g., active status, lower privilege). 5. System creates a time-limited session/token scoped to the target user, annotated with initiator metadata and mode constraints. 6. System logs an immutable audit event for impersonation start (including session ID, mode, and reason). 7. System returns the token or redirects the initiator into the target's session context; initiator acts within the session and actions execute as the target user but include initiator audit metadata. 8. Initiator ends impersonation, or token expires; system revokes the session/token. 9. System logs impersonation end details (session ID, initiator, target, timestamps).
Error Scenario	1. Unauthorized initiator: System returns HTTP 403 and logs a failed attempt. 2. Target user not found or inactive: System returns HTTP 404/400 and logs failure. 3. Business rule violation (e.g., target has equal/higher privilege): System returns HTTP 403 and logs reason. 4. Session or audit logging failure: Impersonation aborts, returns an error, and clears partial session state.

2.2.1.5 Specification of the Use Case Delete User Account and Data

Table 2.7: Use Case Specification: Delete User Account and Data

Title	Delete User Account and Data
Summary	Administrator removes a user account by anonymizing personal information and removing non-essential data. Payment records preserved for audit.
Actor	Administrator
Precondition	Administrator is authenticated and authorized. Target user exists and is not an administrator. Administrator is not deleting own account.
Post-condition	Account removed. Personal information anonymized. Credentials revoked. Non-essential data removed. Payment history preserved. Deletion audited.
Post-condition (Failure)	No changes made. System reports failure and provides guidance.
Trigger	Administrator confirms deletion via the admin UI.
Basic Scenario	1. Administrator locates target account. 2. Administrator selects user and chooses *Delete Account*. 3. System displays confirmation dialog with deletion details. 4. Administrator confirms deletion. 5. System verifies target is not an administrator and not self. 6. System anonymizes personal information. 7. System removes credentials and session tokens. 8. System removes non-essential data. 9. System preserves financial records. 10. System records deletion in audit logs. 11. System displays success and removes user from active list.
Error Scenario	1. If administrator lacks permission, system rejects request and displays authorization error. 2. If target user is an administrator or administrator attempts to delete own account, system prevents deletion and displays error. 3. If target user does not exist, system displays error and does not proceed. 4. If deletion fails due to system errors, system rolls back changes, preserves user record, and displays error. 5. If partial cleanup fails, system preserves core deletion but flags incomplete cleanup for manual review.

2.2.2 Refinement of the Use Case Manage Infrastructure and Hardware

Figure 2.4 presents the refinement of the Use Case Manage Infrastructure and Hardware for the Administrator actor. This use case includes the following sub-use cases:

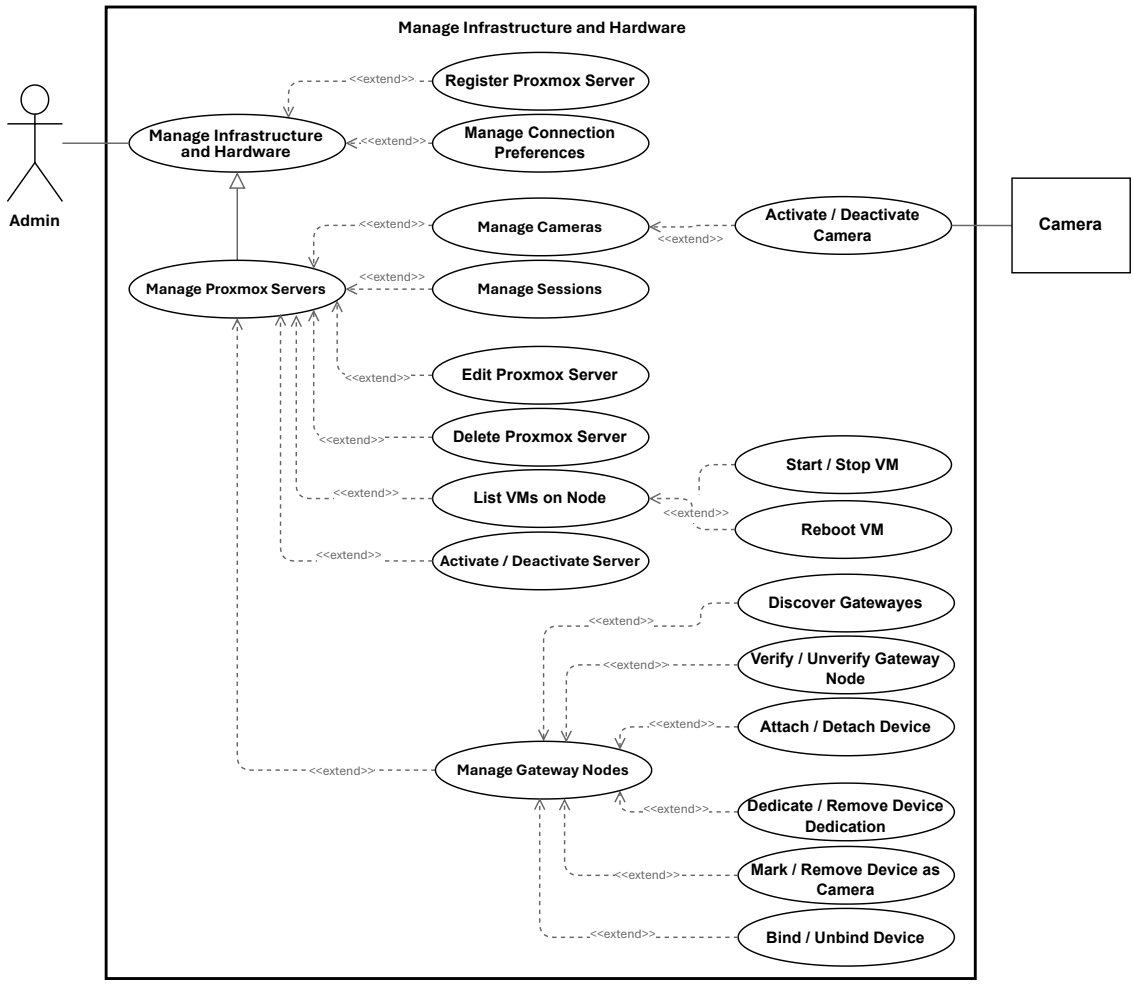


Figure 2.4: Use Case Manage Infrastructure and Hardware

2.2.2.1 Specification of the Use Case Register Proxmox Server

Table 2.8: Use Case Specification: Register Proxmox Server

Title	Register Proxmox Server
Summary	Administrator registers a new Proxmox VE server to enable VM provisioning. System validates connectivity and discovers nodes before adding server to inventory.
Actor	Administrator
Precondition	Administrator is authenticated and authorized. Network access to Proxmox server and valid credentials are available.
Post-condition	Proxmox server is registered and appears in inventory. Discovered nodes are available for provisioning. Configuration is securely stored.
Basic Scenario	1. Administrator opens infrastructure management and selects Register New Server. 2. Administrator enters server details: host address, port, credentials, and optional resource configuration (e.g., VM ID ranges). 3. System validates submitted information. 4. System attempts to connect to Proxmox server using provided credentials. 5. System discovers available cluster nodes. 6. System registers server and stores configuration securely. 7. System displays registered server in inventory with discovered nodes. 8. Administrator can now use this server for provisioning.
Error Scenario	1. If submitted information is invalid or incomplete, system rejects request and displays validation error. 2. If system cannot connect to Proxmox server (unreachable, invalid credentials), system displays error and does not register server. 3. If server is already registered, system informs administrator and suggests updating existing entry. 4. If no cluster nodes are discovered, system flags this and allows administrator to review configuration before proceeding. 5. If system error occurs during registration, system displays error and leaves server unregistered.

2.2.2.2 Specification of the Use Case Edit Proxmox Server

Table 2.9: Use Case Specification: Edit Proxmox Server

Title	Edit Proxmox Server
Summary	Administrator updates the details of an existing Proxmox VE server so the infrastructure inventory remains accurate and usable for provisioning.
Actor	Administrator
Precondition	Administrator is authenticated and authorized. The target Proxmox server exists in the inventory.
Post-condition	The Proxmox server details are updated in the inventory and available for subsequent infrastructure management tasks.
Basic Scenario	1. Administrator opens the server details page and selects the edit option. 2. System displays the current server information. 3. Administrator modifies one or more server details. 4. Administrator submits the changes. 5. System validates the submitted information. 6. System updates the server record. 7. System confirms successful update and shows the revised server details.
Error Scenario	1. Validation failure: system rejects incomplete or invalid input and preserves the entered values for correction. 2. Unauthorized access: system denies the request if the Administrator is not permitted to edit the server. 3. Server not found: system informs the Administrator that the selected server is unavailable. 4. Update failure: system informs the Administrator that the changes could not be saved and keeps the original data unchanged.

2.2.2.3 Specification of the Use Case Delete Proxmox Server

Table 2.10: Use Case Specification: Delete Proxmox Server

Title	Delete Proxmox Server
Summary	Administrator removes a registered Proxmox VE server from the infrastructure inventory when it is no longer needed.
Actor	Administrator
Precondition	Administrator is authenticated and authorized to manage infrastructure. The target Proxmox server exists in the inventory.
Post-condition	The Proxmox server is removed from the inventory and is no longer available for provisioning.
Basic Scenario	1. Administrator selects a Proxmox server and chooses Delete. 2. System asks the Administrator to confirm the deletion. 3. Administrator confirms the deletion. 4. System checks that the server can be deleted. 5. System removes the server from the inventory. 6. System confirms successful deletion.
Error Scenario	1. Deletion canceled: Administrator closes the dialog or selects Cancel; no action is taken. 2. Server not eligible for deletion: system informs the Administrator that the server cannot be removed. 3. Server not found: system informs the Administrator that the selected server is unavailable. 4. Unauthorized access: system denies the request if the Administrator is not permitted to delete infrastructure entries. 5. Deletion failure: system informs the Administrator that the server could not be deleted and keeps the current state unchanged.

2.2.2.4 Specification of the Use Case Manage Connection Preferences

Table 2.11: Use Case Specification: Manage Connection Preferences

Title	Manage Connection Preferences
Summary	Administrator creates, updates, tests, or deletes connection preferences (endpoints, ports, protocols, and credentials) for external platform services (Proxmox, Guacamole, MQTT, Gateway). Preferences are stored securely, auditable, and optionally tested for connectivity.
Actor	Administrator
Precondition	Administrator authenticated and authorized; SecretStore available; UI or API accessible; network access to target service if testing is requested.
Post-condition	Preference persisted with secrets stored securely (encrypted); audit record created; test result recorded if requested; dependent processes (e.g., node discovery) triggered when applicable.
Basic Scenario	1. Administrator opens the Connection Preferences editor and submits a create or update request. 2. Controller validates input and forwards to Service. 3. Service encrypts credentials via SecretStore and persists metadata to the repository. 4. If requested, Service invokes an ExternalClient to test connectivity and records the result. 5. Service writes an audit entry (redacting secrets) and returns the persisted preference and test outcome. 6. Controller responds to the Administrator with success and updated resource.
Error Scenario	1. Validation failure: Controller returns HTTP 422; no changes persisted. 2. Unauthorized: Controller returns HTTP 403; attempt logged. 3. SecretStore/repository failure: Service rolls back and returns HTTP 500. 4. Connection test failure: Service records failed test and returns details; persistence may be allowed or rejected per policy.

2.2.2.5 Specification of the Use Case Discover Gateway Nodes

Table 2.12: Use Case Specification: Discover Gateway Nodes

Title	Discover Gateway Nodes
Summary	Administrator initiates a discovery scan to find all available gateway nodes on the registered Proxmox infrastructure and checks their online status.
Actor	Administrator
Precondition	Administrator is authenticated and authorized to manage infrastructure. At least one Proxmox server is registered and reachable.
Post-condition	System identifies all available gateway nodes in the infrastructure. Each discovered node's online/offline status is verified. A list of discovered nodes with their status is displayed to the administrator.
Basic Scenario	1. Administrator opens the infrastructure management interface. 2. Administrator initiates the *Discover Gateway Nodes* operation. 3. System queries all registered Proxmox servers for available gateway nodes. 4. System checks the online status of each discovered node. 5. System displays a list of discovered nodes, including their names, IP addresses, and current status (online/offline). 6. Administrator can review the list and register newly discovered nodes or update existing entries.
Error Scenario	1. If the administrator lacks permission, the request is denied and an authorization error is displayed. 2. If no Proxmox servers are registered, the system informs the administrator and does not attempt discovery. 3. If a Proxmox server is unreachable during discovery, the system skips it and continues with remaining servers. 4. If no gateway nodes are found, the system displays an empty result and suggests verifying the infrastructure setup. 5. If the discovery operation fails due to a system error, the system displays an error message with any partial results found before the failure.

2.2.2.6 Specification of the Use Case Verify / Unverify Gateway Node

Table 2.13: Use Case Specification: Verify / Unverify Gateway Node

Title	Verify / Unverify Gateway Node
Summary	Allows an administrator to mark a gateway node as trusted (verified) or revoke its trusted status (unverified). The system records verification information and may perform a connectivity test before completing the operation.
Actor	Administrator
Precondition	The administrator is authenticated and authorized. The target gateway node exists in the system.
Post-condition	The verification status of the gateway node is successfully updated and an audit record is created.
Basic Scenario	1. The administrator accesses the gateway inventory. 2. The administrator selects a gateway node. 3. The administrator selects the Verify or Unverify option. 4. The system validates the request. 5. The system performs the connectivity test if requested. 6. The system updates the verification status and stores the verification information. 7. The system records the operation in the audit log. 8. The system displays the updated gateway node information.
Error Scenario	1. **Unauthorized Access:** The system rejects the request and informs the administrator. 2. **Validation Error:** The submitted information is invalid and the system requests correction. 3. **Connectivity Test Failure:** The connectivity test fails and the system notifies the administrator. 4. **Gateway Node Not Found:** The selected gateway node does not exist. 5. **System Error:** An unexpected error occurs and the operation is cancelled.

2.2.2.7 Specification of the Use Case Dedicate / Remove Device Dedication

Table 2.14: Use Case Specification: Dedicate / Remove Device Dedication

Title	Dedicate / Remove Device Dedication
Summary	Administrator permanently assigns a USB device to a specific VM for automatic attachment when sessions are created on that VM, or revokes the assignment.
Actor	Administrator
Precondition	Administrator is authenticated and authorized to manage hardware. The USB device exists in the inventory. The target Proxmox server and VM are valid and reachable.
Post-condition	1. (Dedicate) The device is bound to the target VM. Any current attachment or gateway binding is cleared. The device is ready for automatic attachment to future sessions on that VM. Any linked camera is assigned to the VM. 2. (Remove Dedication) The device is no longer bound to any specific VM. Any linked camera assignment is cleared.
Basic Scenario	1. Administrator navigates to the device details in the admin interface. 2. Administrator selects either *Dedicate to VM* or *Remove Dedication*. 3. For dedication: Administrator specifies the target Proxmox server and VM. 4. System validates the request and checks that the device is not actively in use. 5. System applies the dedication (or removes it) and confirms success. 6. System displays the updated device status and binding information.
Error Scenario	1. If the administrator lacks permission, the request is rejected and an authorization error is displayed. 2. If the specified VM or server does not exist, the system rejects the request and asks for valid input. 3. If the device is currently attached to an active session, the system prevents the operation and instructs the user to detach it first. 4. If the operation fails (e.g., due to unavailable services), the system displays an error and leaves the device in its previous state.

2.2.2.8 Specification of the Use Case Activate / Deactivate Camera

Table 2.15: Use Case Specification: Activate / Deactivate Camera

Title	Activate / Deactivate Camera
Summary	Administrator or engineer in an active session activates a robot camera to enable live streaming, or deactivates it to stop the stream.
Actor	Administrator, Engineer (in active session)
Precondition	Actor is authenticated and authorized to control cameras. Target camera exists and is connected to an online robot. For activation, the camera must not already be streaming.
Post-condition	If activated: Camera begins streaming; live video feed is available to authorized users. If deactivated: Camera stops streaming; video feed is no longer available. The action is recorded for audit.
Basic Scenario	1. Actor opens the camera control interface. 2. Actor selects a camera from the available list. 3. Actor chooses either *Start Stream* or *Stop Stream*. 4. System validates the request and checks that the camera and robot are reachable. 5. System sends the command to the robot and waits for confirmation. 6. Robot acknowledges the request and toggles the camera. 7. System verifies the stream is active or stopped as expected. 8. System displays the updated camera status to the actor. 9. If streaming was activated, the live feed is now available in the interface.
Error Scenario	1. If the actor lacks permission to control cameras, the request is denied and an authorization error is displayed. 2. If the camera or robot is not reachable, the system notifies the actor and does not attempt the operation. 3. If the robot does not respond to the command within a reasonable timeout, the system reports the failure and suggests checking robot connectivity. 4. If the camera is already in the requested state (e.g., already streaming when activation is requested), the system displays a notice. 5. If the stream cannot be established after the robot confirms, the system displays an error and reverts the robot camera state if possible.

2.2.2.9 Specification of the Use Case Manage Sessions

Table 2.16: Use Case Specification: Manage Sessions

Title	Manage Sessions
Summary	Administrator manages VM sessions: view, extend, or terminate. Actions are auditable and may trigger cleanup (Guacamole, Proxmox, hardware).
Actor	Administrator (primary). System services: VMSessionController, VMSessionCleanupService, ExtendSessionService, TerminateVMJob, GuacamoleClient, ProxmoxClient, GatewayService, CameraService (secondary).
Precondition	Administrator is authenticated and authorised. Target session exists and is manageable. Guacamole/Proxmox services are reachable.
Post-condition	On success: session expiry is updated (extend) or session is ended and resources are released (terminate). Guacamole connections are removed and devices/camera controls are released. Actions are audited.
Trigger	Administrator opens session management UI or selects Extend/Terminate on a session.
Basic Scenario	1. Administrator opens session view. 2. System expires overdue sessions and lists active sessions. 3. Administrator selects a session and views details. 4. Administrator chooses Extend or Terminate. 5. System validates authorization and session state. 6. If extend: system applies rules, updates `expires_at`, and returns updated session. 7. If terminate: system deletes Guacamole connection, releases camera controls and USB attachments, optionally stops or reverts the VM, and marks session as ended. 8. System refreshes the list and displays confirmation.
Error Scenario	1. If unauthorized, system returns HTTP 403 and logs the attempt. 2. If the session is already ended, system returns no-op or validation error. 3. If extension violates quota or time-window rules, system returns validation error and does not update expiry. 4. If termination cleanup fails (Guacamole/Proxmox/Hardware), system logs the error, retries according to job policy, and returns a warning; the session is force-marked as ended for later reconciliation when safe.

2.2.3 Refinement of the Use Case Manage Content Studio

Figure 2.5 presents the refinement of the Use Case Manage Content Studio for the Teacher actor. This use case includes the following sub-use cases:

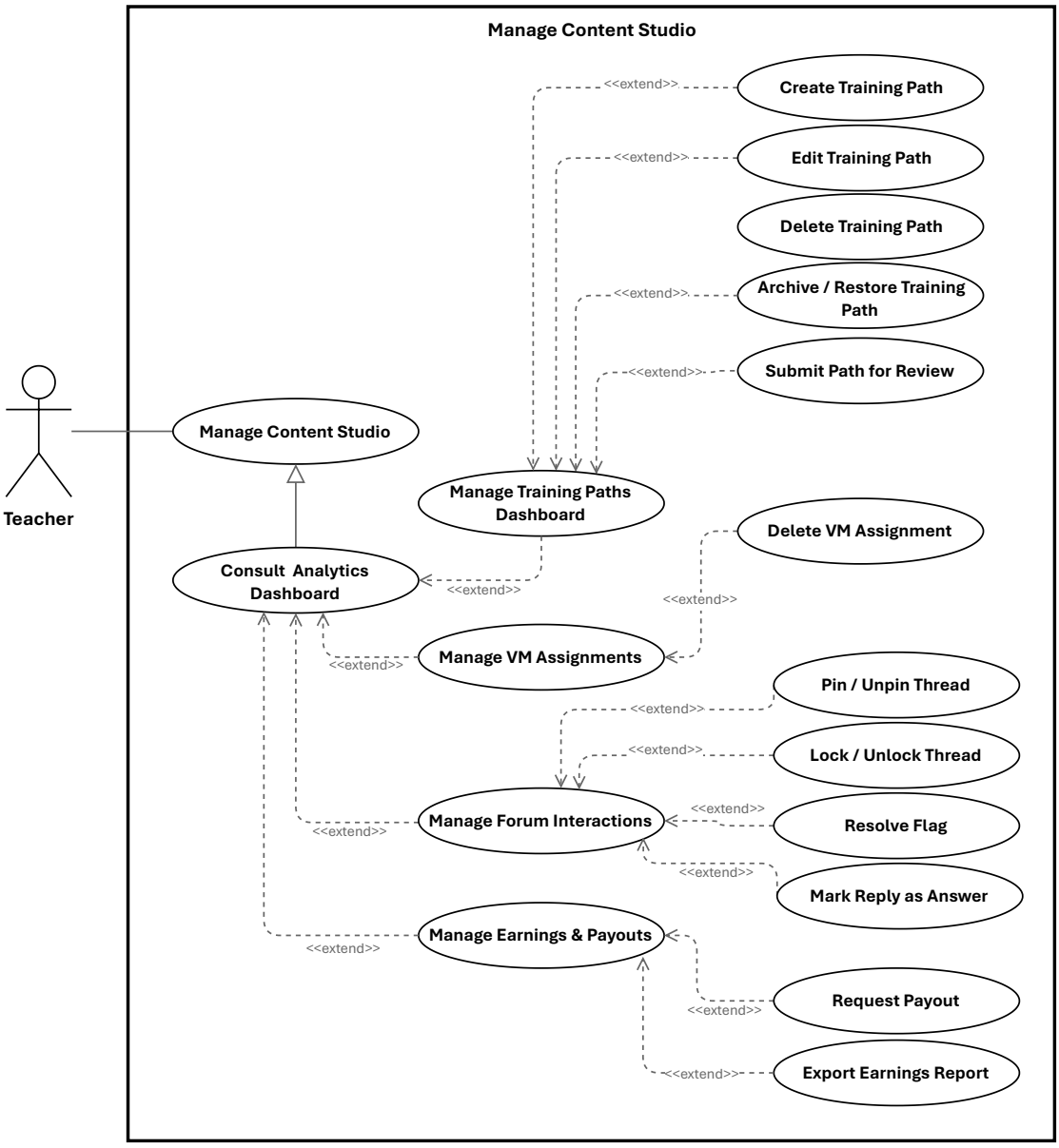


Figure 2.5: Use Case Manage Content Studio

2.2.3.1 Specification of the Use Case Create Training Path

2.2.3.2 Specification of the Use Case Edit Training Path

Table 2.17: Use Case Specification: Edit Training Path

Title	Edit Training Path
Summary	Teacher updates the details of an existing training path so its content, description, and related information remain accurate and aligned with the teaching plan.
Actor	Teacher
Precondition	Teacher is authenticated, owns the target training path, and the path is available for editing.
Post-condition	The training path details are updated and the Teacher can view the revised information. On failure, the original information remains unchanged.
Basic Scenario	1. Teacher opens the training path editor. 2. System displays the current training path information. 3. Teacher modifies one or more editable fields. 4. Teacher submits the changes. 5. System validates the submitted information. 6. System updates the training path. 7. System confirms successful update and shows the revised details.
Error Scenario	1. Validation failure: system rejects incomplete or invalid input and preserves the entered values for correction. 2. Unauthorized access: system denies the request if the Teacher is not allowed to edit the training path. 3. Training path not found: system informs the Teacher that the requested path is unavailable. 4. Update failure: system informs the Teacher that the changes could not be saved and keeps the original data unchanged.

2.2.3.3 Specification of the Use Case Delete Training Path

Table 2.18: Use Case Specification: Delete Training Path

Title	Delete Training Path
Summary	Teacher permanently removes one of their own training paths after confirming the action.
Actor	Teacher
Precondition	Teacher is authenticated, owns the target training path, and the path is eligible for deletion.
Post-condition	The training path is removed and no longer appears in the teacher's active listings.
Basic Scenario	1. Teacher selects a training path and chooses Delete. 2. System asks the Teacher to confirm the deletion. 3. Teacher confirms the deletion. 4. System verifies that the Teacher is allowed to delete the training path. 5. System removes the training path from active listings. 6. System confirms successful deletion.
Error Scenario	1. Deletion canceled: Teacher closes the dialog or selects Cancel; no action is taken. 2. Training path not eligible for deletion: system informs the Teacher that the path cannot be deleted. 3. Training path not found: system informs the Teacher that the requested path is unavailable. 4. Unauthorized access: system denies the request if the Teacher does not own the training path. 5. Deletion failure: system informs the Teacher that the path could not be deleted and keeps the current state unchanged.

2.2.3.4 Specification of the Use Case Archive / Restore Training Path

Table 2.19: Use Case Specification: Archive / Restore Training Path

Title	Archive / Restore Training Path
Summary	Teacher archives a training path to hide it from active listings while preserving content; later restores it to active listings.
Actor	Teacher
Precondition	Teacher is authenticated and owns the training path; it exists in the system.
Post-condition	Archive: path marked archived, removed from active listings, content preserved. Restore: archived flag cleared, path returns to active listings.
Basic Scenario	1. Teacher selects path and chooses "Archive". 2. System asks for confirmation. 3. Teacher confirms. 4. System verifies ownership and archivable state. 5. System marks path as archived, records audit, and removes from active listings. 6. System confirms success; item no longer appears in active lists. 7. Later, Teacher navigates to archived items and chooses "Restore". 8. System asks for confirmation. 9. Teacher confirms. 10. System verifies ownership and archived state, clears archived status, records audit, and restores to active listings. 11. System confirms success; item appears again in active lists.
Error Scenario	1. If Teacher does not own path, system denies action and displays authorization message. 2. If path is not found or already in requested state, system displays validation message. 3. If system persistence error occurs, system preserves consistent state, logs incident, and advises retry. 4. If auxiliary indexing fails, system completes in primary store, records inconsistency for reconciliation, and informs Teacher that listings may update shortly.

2.2.3.5 Specification of the Use Case Delete VM Assignment

Table 2.20: Use Case Specification: Delete VM Assignment

Title	Delete VM Assignment
Summary	Teacher deletes an existing VM assignment they created; the system releases any provisioned VM and remote connections, records the action for audit, and notifies relevant parties.
Actor	Teacher (primary). System (secondary).
Precondition	Teacher is authenticated and authorized to manage the assignment; the VM assignment exists and is eligible for deletion.
Post-condition	On success: the assignment is removed (or soft-deleted per policy); associated external resources are released or scheduled for cleanup; an audit entry is recorded; affected users are notified.
Basic Scenario	1. Teacher locates the VM assignment and chooses Delete. 2. System asks the Teacher to confirm the deletion. 3. Teacher confirms the deletion. 4. System verifies the Teacher's authorization and that the assignment may be deleted. 5. System releases or schedules cleanup of any provisioned VM and removes any remote access/connection resources associated with the assignment. 6. System removes (or marks as deleted) the assignment record and records the deletion in the audit log. 7. System notifies the Teacher and any other relevant parties about the deletion and cleanup status. 8. The UI reflects the deletion and updated resource status.
Error Scenario	1. Assignment not found: system informs the Teacher and makes no changes. 2. Unauthorized (not owner or insufficient privileges): system denies the action and explains required permissions. 3. External resource cleanup fails (e.g., VM removal or remote connection deletion): system logs the failure, retries or enqueues a cleanup job, may perform a soft-delete of the assignment per policy, and notifies the Teacher of the partial outcome and next steps. 4. System error during deletion: system preserves consistent state, records the incident for investigation, and advises the Teacher to retry later or contact support.

2.2.3.6 Specification of the Use Case Pin / Unpin Thread

Table 2.21: Use Case Specification: Pin / Unpin Thread

Title	Pin / Unpin Thread
Summary	Teacher pins or unpins a forum thread within a training path to highlight or remove emphasis. The system updates the thread's visibility and records the moderation action for audit.
Actor	Teacher (primary)
Precondition	Teacher is authenticated and has moderation permission for the training path; the target thread exists and is not deleted.
Post-condition	The thread's pinned state is updated and persisted; forum listings and thread views reflect the change; a moderation audit entry is recorded.
Basic Scenario	1. Teacher opens the forum inbox or thread detail and selects the target thread. 2. Teacher chooses the Pin or Unpin action. 3. System verifies the teacher's authorization and that the thread is in a valid state for moderation. 4. System updates the thread's pinned status and persists the change. 5. System records a moderation audit entry (actor, action, thread identifier, timestamp). 6. System updates forum listings and thread ordering so the change is visible to authorized users. 7. UI displays confirmation and the thread's new pinned state.
Error Scenario	1. Unauthorized: If the teacher lacks moderation permission, the system rejects the action and informs the teacher. 2. Not found / invalid state: If the thread is missing or deleted, the system reports the condition and takes no action. 3. Persistence failure: If the update cannot be saved, the system aborts the change, logs the failure, records an error audit, and notifies the teacher to retry later.

2.2.3.7 Specification of the Use Case Lock / Unlock Thread

Table 2.22: Use Case Specification: Lock / Unlock Thread

Title	Lock / Unlock Thread
Summary	Teacher locks a thread to prevent further replies or unlocks it to allow discussion; the system enforces the state, records a moderation audit entry, and optionally notifies subscribers.
Actor	Teacher (primary)
Precondition	Teacher is authenticated and authorised to moderate the training path containing the thread; the thread exists and is not archived.
Post-condition	On success: the thread's locked state is set (locked/unlocked) and persisted; a moderation audit entry is recorded; subscribers may be notified of the change.
Basic Scenario	1. Teacher selects Lock or Unlock for a target thread and confirms the action. 2. System verifies the teacher's moderation permission and that the thread is in a valid state for the requested action. 3. System updates and persists the thread's locked state. 4. System records a moderation audit entry with actor, action, thread identifier and timestamp. 5. System updates forum listings/permission enforcement so replies are blocked or allowed accordingly. 6. System returns confirmation to the teacher and the UI reflects the updated thread metadata.
Error Scenario	1. Unauthorized: If the teacher lacks moderation permission, the system rejects the action and informs the teacher. 2. Not found / invalid state: If the thread cannot be located or is archived, the system reports the condition and takes no action. 3. Persistence / notification failure: If the locked-state update or notification cannot be saved/sent, the system aborts or rolls back the change, logs the failure, records an error audit, and informs the teacher to retry later.

2.2.3.8 Specification of the Use Case Resolve Flag

Table 2.23: Use Case Specification: Resolve Flag

Title	Resolve Flag
Summary	Teacher reviews a flagged discussion thread within an owned training path, takes an appropriate moderation action, and clears the flag so the thread is no longer treated as requiring attention.
Actor	Teacher
Precondition	Teacher is authenticated and owns the training path containing the flagged thread; the target thread exists and is currently flagged.
Post-condition	The flag is cleared on the target thread and persisted; the thread is removed from flagged-content views; the action is recorded for audit and the teacher receives confirmation.
Basic Scenario	1. Teacher opens the forum inbox or flagged-content view. 2. System displays flagged threads belonging to the teacher's training paths. 3. Teacher selects a flagged thread and reviews the content. 4. Teacher chooses the Resolve Flag action. 5. System verifies ownership and that the thread is still flagged. 6. System clears the flag, persists the updated moderation state, and records an audit entry. 7. System confirms success and the thread is removed from flagged-content views.
Error Scenario	1. Unauthorized: If the teacher does not own the training path, the system rejects the action and keeps the flag unchanged. 2. Invalid state: If the thread is no longer flagged or cannot be found, the system informs the teacher and makes no change. 3. Persistence failure: If the unflag operation cannot be saved, the system preserves the previous state, logs the failure, records an error audit, and prompts the teacher to retry later.

2.2.3.9 Specification of the Use Case Request Payout

Table 2.24: Use Case Specification: Request Payout

Title	Request Payout
Summary	Teacher requests withdrawal of an amount from available earnings. The system validates amount and payment method according to payout policies, creates a pending payout request, and confirms submission.
Actor	Primary: Teacher. Secondary: System.
Precondition	Teacher is authenticated and has earnings data with a non-zero available balance. Earnings/payout interface is available.
Post-condition	Payout request is created with status pending and associated with the teacher account. System confirms successful submission.
Basic Scenario	1. Teacher opens the earnings section and selects Request Payout. 2. System displays payout form with available balance and payment options. 3. Teacher enters payout amount and selects payment method. 4. Teacher submits the request. 5. System validates fields and checks available balance and minimum threshold. 6. System verifies amount and payment method validity. 7. System creates payout request with pending status and confirms submission to the Teacher.
Error Scenario	1. Amount exceeds available balance: system rejects request and displays a validation message; Teacher may retry with a valid amount. 2. Amount below minimum threshold: system rejects request and prompts Teacher to increase amount; Teacher may retry. 3. Invalid payment method: system rejects request and requests a valid method; Teacher may retry. 4. Persistence or service failure: system preserves previous state, logs the incident, records an error audit, and informs the Teacher to retry later.

2.2.3.10 Specification of the Use Case Export Earnings Report

Table 2.25: Use Case Specification: Export Earnings Report

Title	Export Earnings Report
Summary	Teacher exports a CSV report of completed earnings transactions for a selected date range so the income history can be reviewed, reconciled, or archived for accounting purposes. The system generates a downloadable file without changing any financial records.
Actor	Primary: Teacher. Secondary: System.
Precondition	Teacher is authenticated and can access the earnings page. Earnings data is available in the system. If a date range is provided, it is valid.
Post-condition	CSV report is generated for the teacher's completed payments within the selected date range. Browser receives streamed download named earnings-YYYY-MM-DD.csv. No financial data is modified.
Basic Scenario	1. Teacher opens the earnings page and requests an export. 2. System resolves the authenticated teacher and determines the export date range (selected or default). 3. System generates a CSV from the teacher's completed payments for the date range. 4. System streams the CSV file to the teacher's browser and the download starts.
Error Scenario	1. Invalid or empty date range: supplied date range is malformed or inverted; system rejects the request and displays a validation message; no download occurs. 2. Export processing failure: data retrieval or CSV generation fails; system aborts the export, preserves current state, logs the incident, and surfaces a recoverable error. 3. No matching payments: selected date range contains no completed payments; system generates a CSV with header row only and streams it successfully.

2.2.4 Refinement of the Use Case Manage Enrolled Training Paths

Figure 2.6 presents the refinement of the Use Case Manage Enrolled Training Paths for the Engineer actor. This use case includes the following sub-use cases:

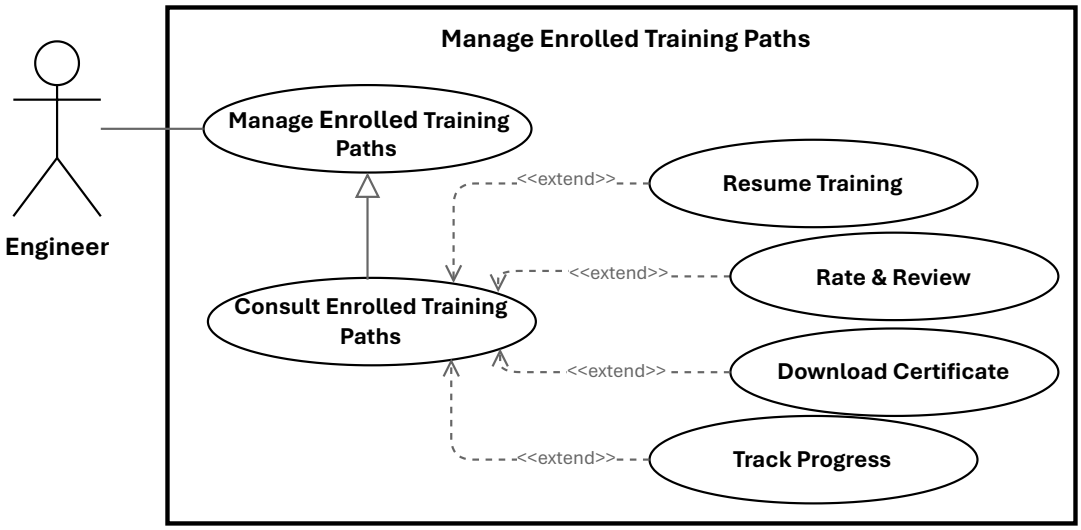


Figure 2.6: Use Case Manage Enrolled Training Paths

2.2.4.1 Specification of the Use Case Resume Training

Table 2.26: Use Case Specification: Resume Training

Title	Resume Training
Summary	Learner resumes a previously started training unit or module at the last saved position so they can continue without losing progress.
Actor	Primary: Engineer (Learner). Secondary: System.
Precondition	Learner is authenticated and enrolled in the training path; the platform stores progress for the unit.
Post-condition	On success: learner resumes at the saved position and the system continues tracking and persisting progress.
Basic Scenario	1. Learner clicks “Resume” on the training unit page. 2. System verifies the learner’s authentication status. 3. System verifies that the learner is enrolled and has access to the training unit. 4. System retrieves the last saved progress for the selected unit. 5. System loads the content metadata and associated learning resource. 6. System returns the resume details, including the content location and the starting position. 7. Learner starts playback or continues reading from the saved position. 8. System continues to record progress updates as the learner resumes training.
Error Scenario	1. Authentication failure: session is invalid or expired; the system blocks access and requests re-authentication. 2. Content retrieval failure: the learning resource cannot be loaded; the system displays an error and may allow retry. 3. Progress persistence failure: the system cannot save updates; the previous state is preserved and a recoverable error is shown.

2.2.4.2 Specification of the Use Case Rate & Review

Table 2.27: Use Case Specification: Rate & Review

Title	Rate & Review
Summary	Engineer submits a star rating and optional written review for a training path. The system validates the submission, stores the review, recalculates the training path's average rating, and applies moderation rules when required.
Actor	Engineer
Precondition	Engineer is authenticated and has access to the training path review interface; the target training path exists.
Post-condition	On success: the review is saved or updated, the training path average rating is recalculated, and the appropriate visibility or approval state is set.
Post-condition (Failure)	No review is stored or updated; the previous state is preserved and an error is reported.
Basic Scenario	1. Engineer opens the review form for a training path. 2. Engineer selects a star rating and optionally enters review text, then submits the form. 3. System validates the engineer's eligibility and the submitted input. 4. System saves the review and recalculates the training path's average rating. 5. If an existing review is detected, the system updates the existing record. 6. System confirms the submission, or marks it as pending if moderation is required.
Error Scenario	1. Missing rating: the system rejects the submission with a validation error. 2. Persistence failure: the system aborts the save, preserves the previous state, logs the error, and notifies the engineer. 3. Moderation required: the system stores the review as pending and hides it until approval.

2.2.4.3 Specification of the Use Case Download Certificate

Table 2.28: Use Case Specification: Download Certificate

Title	Download Certificate
Summary	Engineer requests and downloads a completion certificate (PDF) for a training path they have completed. The system verifies eligibility, retrieves or generates the certificate, and returns it as a downloadable file while logging the action.
Actor	Primary: Engineer. Secondary: System.
Precondition	The engineer is authenticated; the engineer has completed the training path and a certificate record exists or can be generated.
Post-condition	The engineer receives the certificate file; a download audit record is created; certificate delivery metadata is recorded.
Basic Scenario	1. Engineer requests a certificate download. 2. System authenticates the request and identifies the engineer. 3. System verifies that the engineer is eligible to download the certificate. 4. System checks whether a certificate file already exists. 5. If a file exists, the system retrieves it; otherwise, the system generates the certificate and stores it. 6. System returns the certificate file or a download link to the engineer. 7. The download starts in the browser or client application. 8. System records the successful download in the audit log.
Error Scenario	1. Unauthenticated request: the system denies access and requests re-authentication. 2. Not eligible: the system rejects the request if the engineer is not entitled to the certificate. 3. Certificate generation or storage error: the system aborts the operation, preserves the current state, and logs the failure.

2.2.5 Refinement of the Use Case Manage Virtual Lab

Figure 2.7 presents the refinement of the Use Case Manage Virtual Lab for the Engineer actor. This use case includes the following sub-use cases:

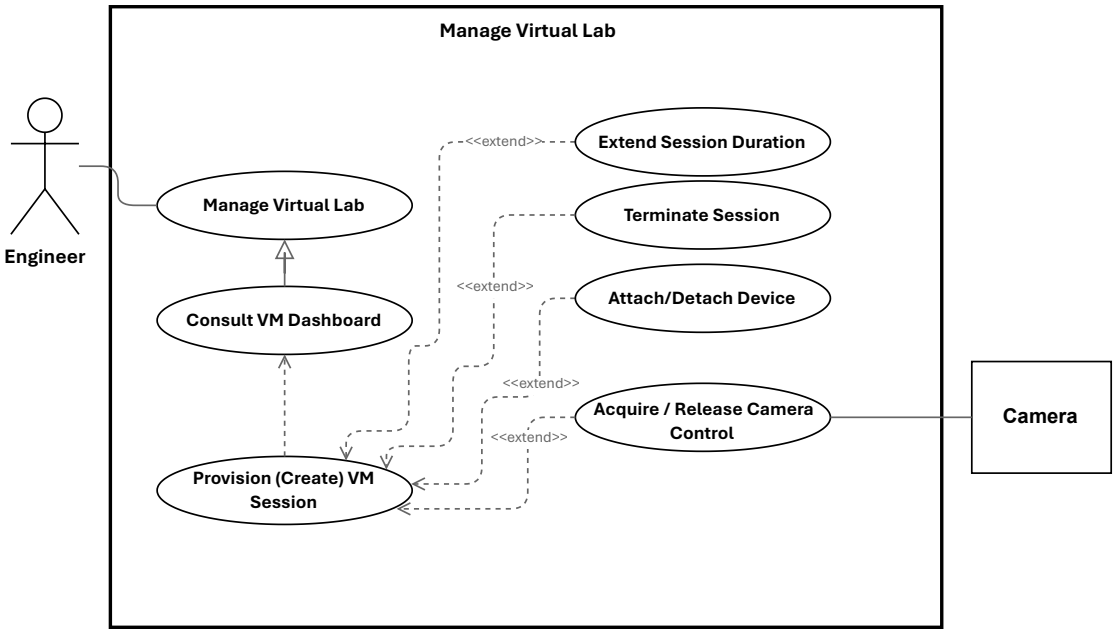


Figure 2.7: Use Case Manage Virtual Lab

2.2.5.1 Specification of the Use Case Provision (Create) VM Session

Table 2.29: Use Case Specification: Provision (Create) VM Session

Title	Provision (Create) VM Session
Summary	Engineer requests an ephemeral VM session by selecting a template and duration. The system validates the request, checks quota, provisions a virtual machine, creates remote access, persists the session, and schedules cleanup.
Actor	Engineer
Precondition	Engineer is authenticated and authorized; the requested template exists and is available; the virtualization and remote-access services are reachable; background processing is operational.
Post-condition	A VM session record exists with status active or provisioning; the VM is provisioned with a host and IP address; remote-access details are created and associated; a cleanup job is scheduled.
Basic Scenario	1. Engineer submits a VM session request with a template and duration. 2. System validates the request and authorizes the engineer. 3. System checks whether the engineer can create a new session. 4. System selects an available node for provisioning. 5. System provisions the virtual machine from the selected template. 6. System waits until provisioning completes and retrieves the VM details. 7. System creates and stores the VM session record. 8. System creates the remote-access connection and associates it with the session. 9. System updates the session status to active and schedules cleanup after the requested duration. 10. System returns the created VM session to the engineer.
Error Scenario	1. Quota exceeded: the system rejects the request and displays an explanatory message. 2. Provisioning failure: the system aborts the operation, attempts cleanup, and marks the session as failed. 3. Persistence failure: the system attempts to roll back the created VM and reports an error. 4. Remote-access setup failure: the system records a partial failure and either schedules remediation or rolls back according to policy.

2.2.5.2 Specification of the Use Case Extend Session Duration

Table 2.30: Use Case Specification: Extend Session Duration

Title	Extend Session Duration
Summary	Engineer requests additional time for an active VM session. The system validates eligibility, checks quota or billing constraints, updates the session expiry, reschedules cleanup, and notifies the engineer.
Actor	Engineer. Secondary: System services.
Precondition	Engineer is authenticated and owns an active VM session; session information is available and the quota or billing service can be consulted.
Post-condition	On success: the session expiry is updated and saved; the cleanup job is rescheduled; quota or billing counters are updated; the engineer is notified and an audit entry is created.
Post-condition (Failure)	No expiry change is made; quota or billing state remains unchanged; the engineer receives an explanatory error and the incident is logged.
Basic Scenario	1. Engineer requests an extension for an active VM session. 2. System authenticates the request and verifies the engineer's ownership of the session. 3. System checks that the session is active and eligible for extension. 4. System verifies quota or billing authorization. 5. If the request is valid, system updates the session expiry, reschedules cleanup, records an audit entry, and notifies the engineer. 6. System returns a success response with the updated session information.
Error Scenario	1. Session not found or not owned: the system rejects the request and returns an authorization or not-found error. 2. Session expired or terminated: the system rejects the request and indicates that a new session must be created. 3. Quota or billing failure: the system rejects the extension request. 4. Persistence or scheduler failure: the system rolls back the update if possible, logs the error, and raises an operational alert.

2.2.5.3 Specification of the Use Case Terminate Session

Table 2.31: Use Case Specification: Terminate Session

Title	Terminate Session
Summary	Engineer ends an active VM session. The system validates authorization, revokes remote access, detaches devices, stops or schedules VM deletion, updates session state, logs the action, and notifies stakeholders.
Actor	Engineer (session owner). Secondary: System.
Precondition	Engineer is authenticated and authorized; the session exists and is in an active state.
Post-condition	On success: the session is marked terminated, remote access is revoked, attached devices are released, the VM is deleted or scheduled for cleanup, an audit entry is recorded, and notifications are sent.
Basic Scenario	1. Engineer issues a terminate request for the session through the UI or API. 2. System authenticates the request and verifies authorization. 3. System marks the session as terminating. 4. System revokes remote access, detaches devices, deletes the VM or schedules cleanup, cancels scheduled jobs, records an audit entry, and notifies stakeholders. 5. System returns a success response with the updated session state.
Error Scenario	1. Authorization failure: the system returns 403 and no change is applied. 2. Persistence failure: the system aborts the operation, logs the error, and returns 500. 3. External API failure: Proxmox or Guacamole errors are retried per policy; persistent failures mark the session with cleanup errors and notify operators.

2.2.5.4 Specification of the Use Case Acquire / Release Camera Control

Table 2.32: Use Case Specification: Acquire / Release Camera Control

Title	Acquire / Release Camera Control
Summary	Engineer acquires exclusive PTZ control of a laboratory camera and later releases it so other users may use the device. The system manages leases, queues, and notifications to coordinate shared access.
Actor	Primary: Engineer. Secondary: Other engineers/users, System Administrator (override), NotificationService, CameraDevice.
Precondition	Engineer is authenticated, has an active session, and the camera is online with PTZ capabilities; control-state tracking is available.
Post-condition	On success: ownership is assigned with a lease expiry; PTZ commands are relayed and acknowledged; on release or lease expiry the camera becomes available and subscribers are notified.
Basic Scenario	1. Engineer requests control of camera C. 2. System verifies authentication, session, and device capabilities. 3. If available, system reserves control and issues a lease with an expiry. 4. System returns success; UI enables PTZ controls and broadcasts a ControlGranted notification. 5. While holding control, the engineer sends PTZ commands which the system relays to the device and acknowledges. 6. Engineer releases control or lease expires; system clears ownership, marks the camera available, and broadcasts ControlReleased.
Error Scenario	1. Camera offline or unreachable: request fails and no ownership change occurs. 2. Unsupported capability: acquire is rejected and PTZ controls remain disabled. 3. Persistence/write failure: system attempts rollback, logs the error, and alerts operations; ownership remains unchanged. 4. Camera busy: system returns BUSY; UI may offer a queue option and the user can re-enter the flow when promoted.

3 Class Diagram

Figure 2.8 presents the class diagram of the IoT-REAP platform.

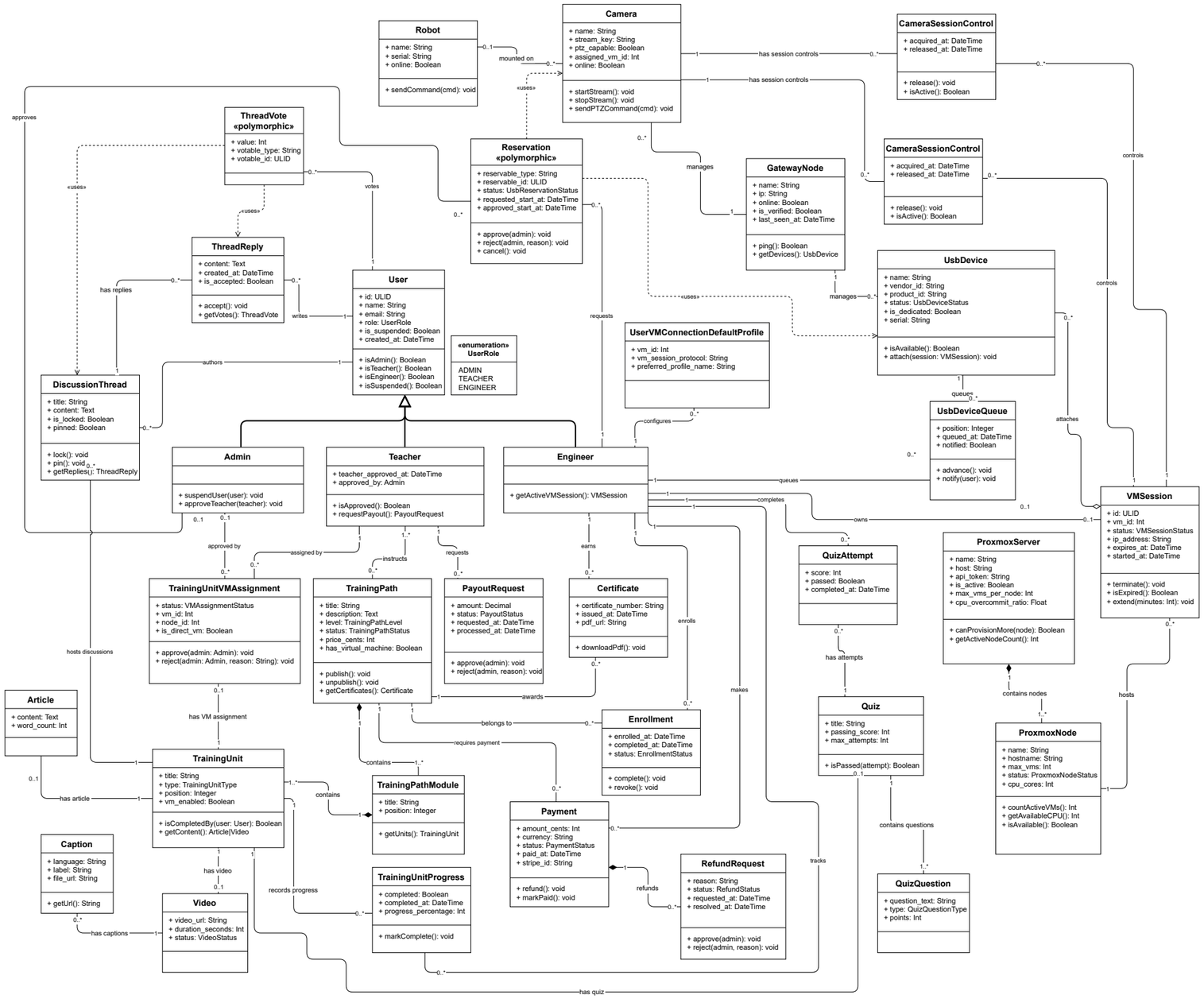


Figure 2.8: Class Diagram of the IoT-REAP Platform

The class diagram encompasses the full domain model of the IoT-REAP platform. The classes are organised into five functional clusters: *user management* (User, Admin, Teacher, Engineer, UserRole), *learning content* (TrainingPath, TrainingPathModule, TrainingUnit and its subtypes, Quiz, Enrollment, Certificate), *infrastructure* (VMSession, ProxmoxServer, ProxmoxNode, GatewayNode, Robot), *hardware resources* (UsbDevice, Camera, Reservation, UsbDeviceQueue), and *financial operations* (Payment, RefundRequest, PayoutRequest). The following paragraphs describe each class individually.

Class User: The class User represents a general platform user and serves as the base entity for all person-like actors. It contains identifying and contact information (such as ULID, name and email), authentication attributes, role membership and suspension flags, and timestamps. User encapsulates shared behaviour and relationships that are common to specialized user types.

Class Admin: The class Admin represents platform administrators with elevated privileges. Admins are responsible for system governance tasks such as user suspension, quota management, teacher approvals, and oversight of infrastructure and financial operations.

Class Teacher: The class Teacher models instructional staff who author and manage training content. Teachers have approval metadata, authoring capabilities, and interfaces for requesting payouts and reviewing learner submissions.

Class Engineer: The class Engineer represents practitioners who consume training materials and operate remote laboratory resources. Engineers interact with VM sessions, hardware reservations, and robot control features.

Class UserRole: An enumeration that defines the possible user roles (ADMIN, TEACHER, ENGINEER). UserRole is used to gate permissions and to differentiate behaviour and available features at runtime.

Class TrainingPath: The class TrainingPath models a purchasable or enrollable course consisting of a sequence of modules and units. It stores metadata such as title, description, level, pricing and publication status.

Class TrainingPathModule: TrainingPathModule groups related TrainingUnit objects into a coherent module or chapter within a TrainingPath. Modules provide ordering, summarization and module-level metadata.

Class TrainingUnit: The class TrainingUnit represents an atomic learning unit (for example a lesson or exercise). A unit may be of various types (Article, Video, Quiz) and tracks progress, assignment status and completion criteria.

Class Article: Article is a content subtype representing textual learning material, including body content, attachments and metadata for rendering.

Class Video: Video is a content subtype representing embedded or hosted video media used within a TrainingUnit.

Class Quiz: The Quiz class models assessment units composed of questions. It references QuizQuestion objects and aggregates attempts and scoring rules.

Class QuizQuestion: QuizQuestion represents individual questions within a Quiz, including question text, possible answers and correct answer metadata.

Class QuizAttempt: QuizAttempt records a learner's attempt at a Quiz, including answers chosen, score achieved and timestamps.

Class DiscussionThread: DiscussionThread models a forum-style discussion attached to a TrainingUnit. It aggregates posts and provides moderation and thread metadata.

Class ThreadReply: ThreadReply represents an individual reply within a DiscussionThread; replies may be accepted, voted on, and linked to their authors.

Class Enrollment: Enrollment links a User to a TrainingPath, capturing enrolment date, progress, and entitlement to access path materials and resources.

Class TrainingUnitProgress: TrainingUnitProgress tracks a learner's status within a specific TrainingUnit (e.g., not started, in progress, completed) and records timestamps and completion evidence.

Class Certificate: Certificate models credential issuance after successful completion of a TrainingPath or assessment; it contains recipient data, issuance date and validation metadata.

Class VMSession: VMSession represents a provisioned virtual machine session used by Engineers. It captures VM identifiers, node allocation, status, expiry time and associations to user reservations, Guacamole connections and cleanup jobs.

Class TrainingUnitVMAssignment: This class links TrainingUnits to VM templates or assignments that require a live virtual machine; it stores assignment status, VM identifiers and approval metadata.

Class UserVMConnectionDefaultProfile: UserVMConnectionDefaultProfile captures per-user connection preferences and default profiles for remote desktop or terminal connections to provisioned VMs.

Class ProxmoxServer: ProxmoxServer represents the Proxmox deployment as a logical entity, containing configuration references, API endpoints and a set of ProxmoxNode objects.

Class ProxmoxNode: ProxmoxNode models an individual compute node within the Proxmox cluster; nodes host VM instances and expose resource metrics used by the scheduler and load balancer.

Class GatewayNode: GatewayNode models infrastructure gateways (for example a dedicated VM or host that proxies hardware or network access) and coordinates services such as USB/IP and camera streaming.

Class UsbDevice: UsbDevice models physical USB devices available for reservation and attachment to VMs, tracking device identifiers, availability status and reservation links.

Class Camera: Camera represents camera hardware in the inventory, including streaming endpoints, supported formats and reservation and attachment state.

Class CameraSessionControl: CameraSessionControl encapsulates session-level control for cameras (attach/detach, format negotiation and stream lifecycle) when cameras are assigned to a VM session.

Class Reservation: Reservation is a polymorphic record representing a requested allocation of a reservable resource (USB device, camera, VM slot). It stores requested and approved time windows, status and references to the reservable entity.

Class UsbDeviceQueue: UsbDeviceQueue models the waiting queue for an unavailable USB device, enabling FIFO assignment and notifications when a device becomes available.

Class Robot: Robot models a remote or simulated robot endpoint that can be reserved or controlled from a VM session; it includes identifiers, command interfaces and telemetry endpoints.

Class Payment: Payment records financial transactions (amount, currency, status, user and training path references) and serves as the source for refund and payout workflows.

Class RefundRequest: RefundRequest models a user-initiated refund claim, including reason, linked payment, eligibility checks and administrative review status.

Class PayoutRequest: PayoutRequest represents teacher-initiated requests to withdraw earned funds; it includes payout amount, recipient details and approval state.

4 Sequence Diagrams

To complement the static view offered by the class diagram, this section presents sequence diagrams for the key workflows of the IoT-REAP platform. The selected scenarios cover the most interaction-intensive and technically critical paths: VM session provisioning and termination, robot reservation and command dispatch, user authentication with session security evaluation, and training unit progression. These diagrams expose the dynamic collaboration between Controllers, Services, external APIs (Proxmox, Guacamole, MQTT broker), and frontend Views, providing the traceability link between the architectural choices described in Section 1 and the concrete implementation presented in Chapter 3.

5 Sequence Diagrams

Sequence diagrams illustrate the temporal interaction and message flow between system actors and components, showing how workflows unfold over time. Activity diagrams model decision points, parallel flows, and state transitions, emphasizing branching conditions and process paths where multiple outcomes are possible. Together, these diagram types capture both the choreography and decision logic of major platform workflows.

5.1 Manage Session Sequence

Figure 2.9 illustrates the manage session sequence.

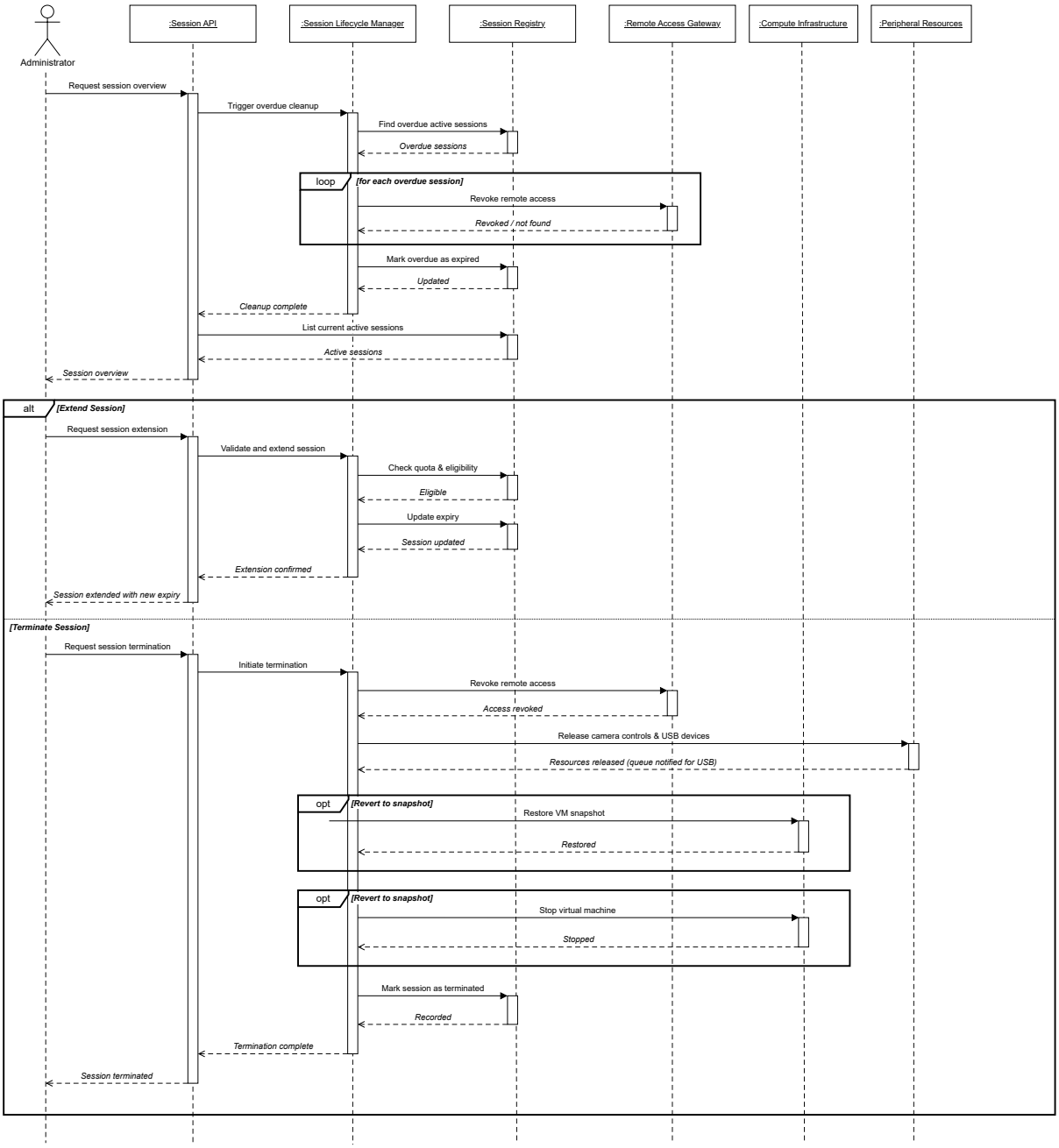


Figure 2.9: Sequence Diagram: Manage Session

5.2 Discover Gateway Node Sequence

Figure 2.10 illustrates the discover gateway node sequence.

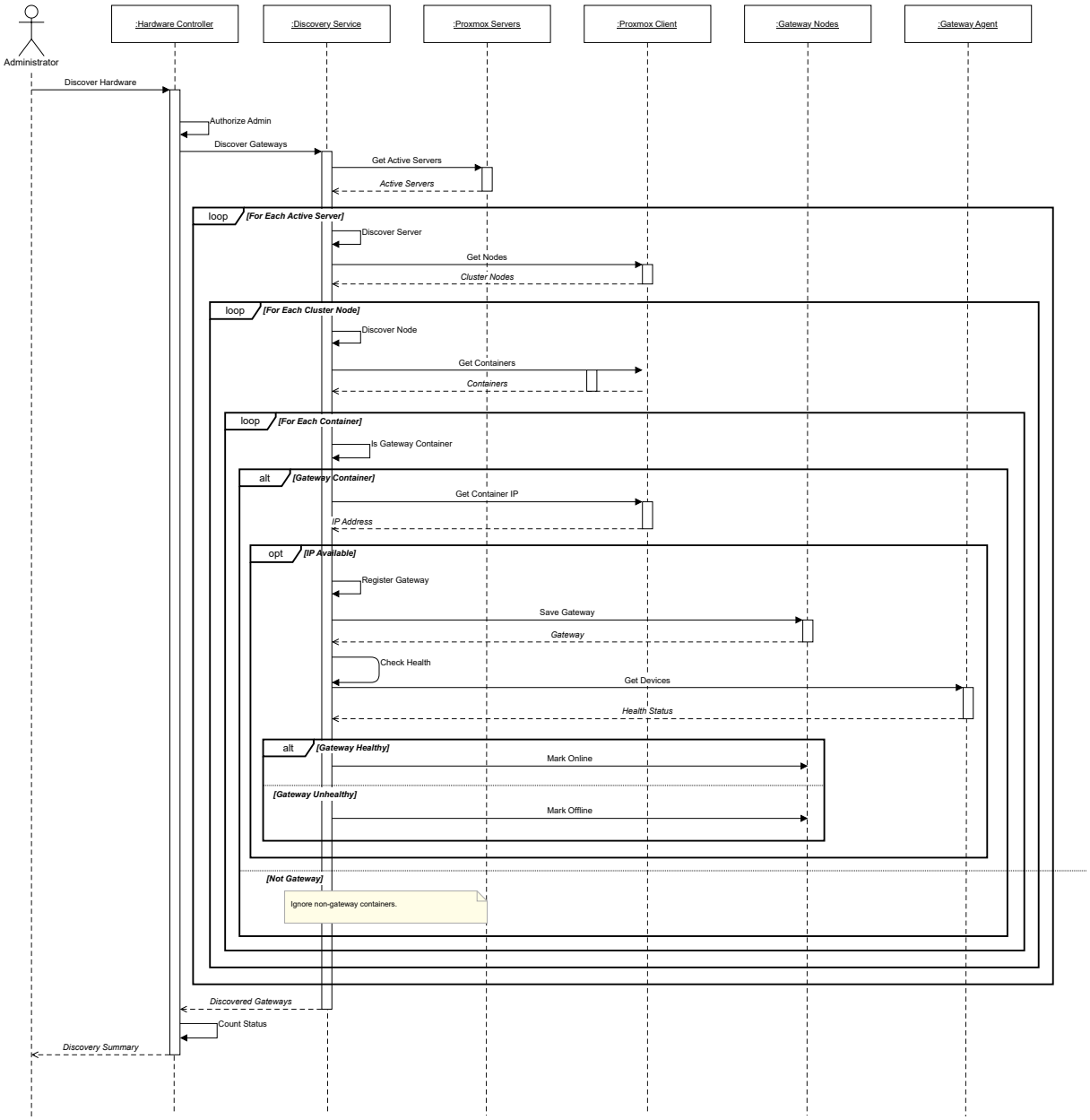


Figure 2.10: Sequence Diagram: Discover Gateway Node

5.3 Payout Request Sequence

Figure 2.11 illustrates the payout request sequence.

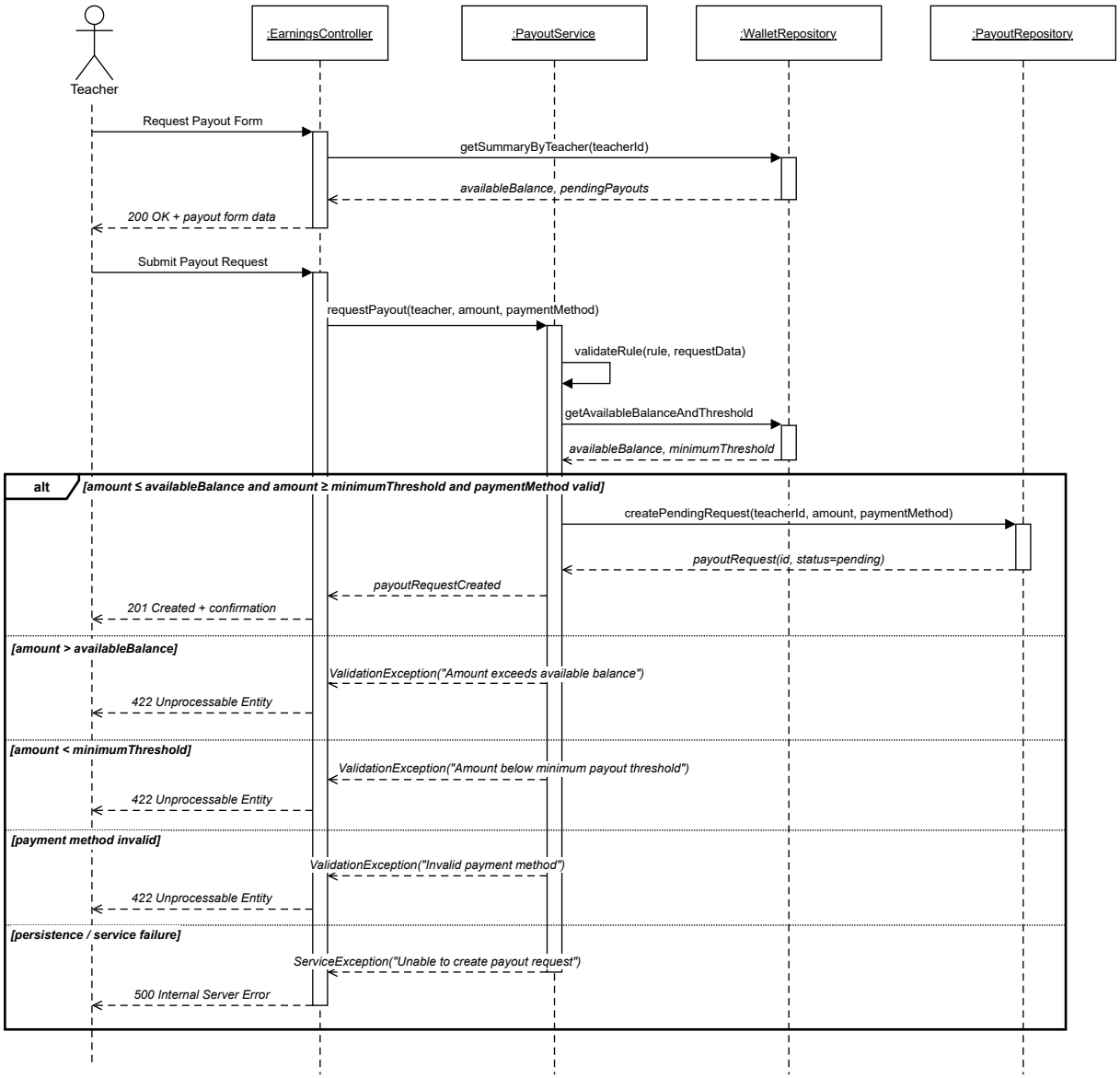


Figure 2.11: Sequence Diagram: Payout Request

5.4 Provision VM Session Sequence

Figure 2.12 illustrates the provision VM session sequence.

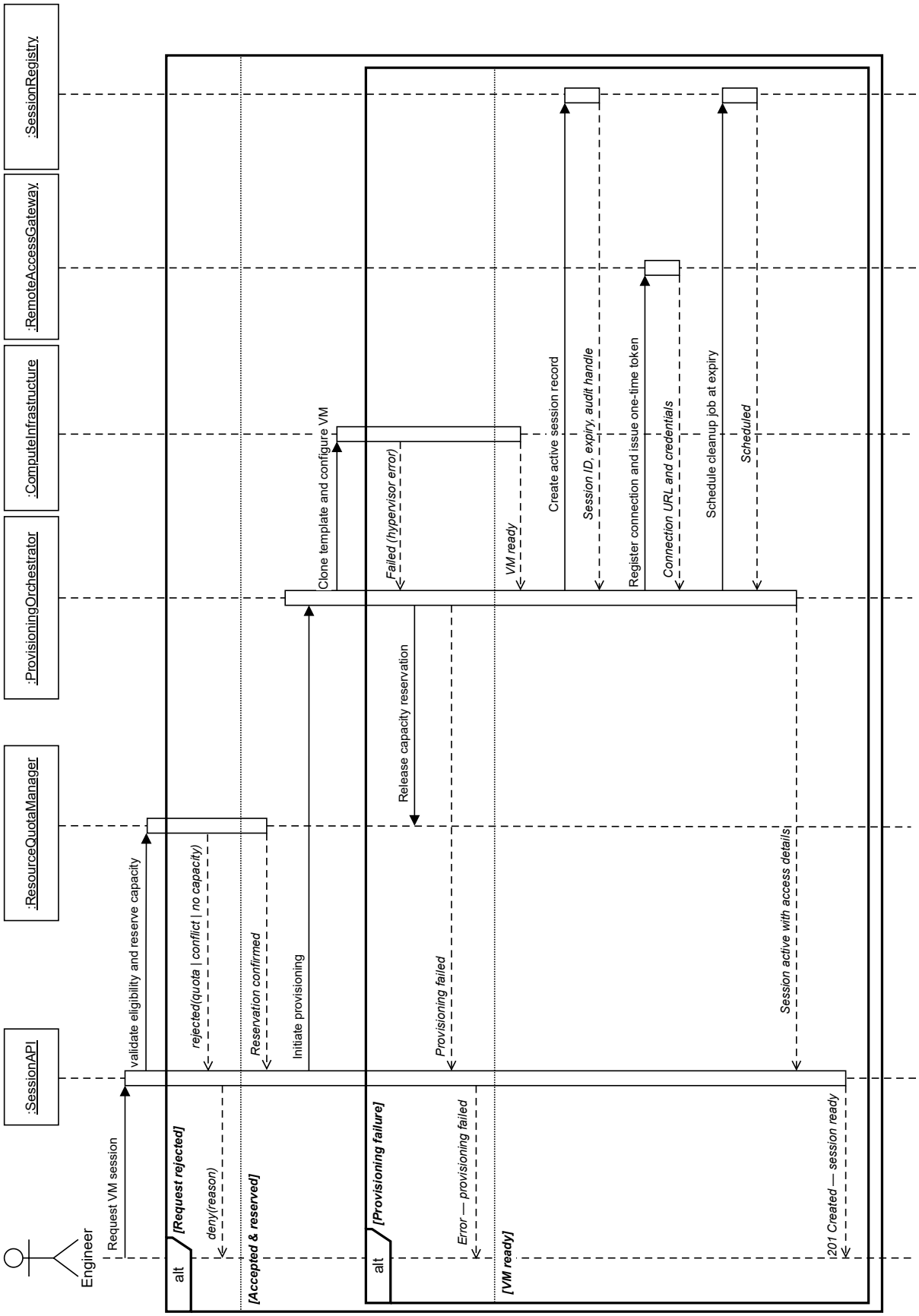


Figure 2.12: Sequence Diagram: Provision VM Session

5.5 User Authentication Sequence

Figure 2.13 illustrates the user authentication sequence.

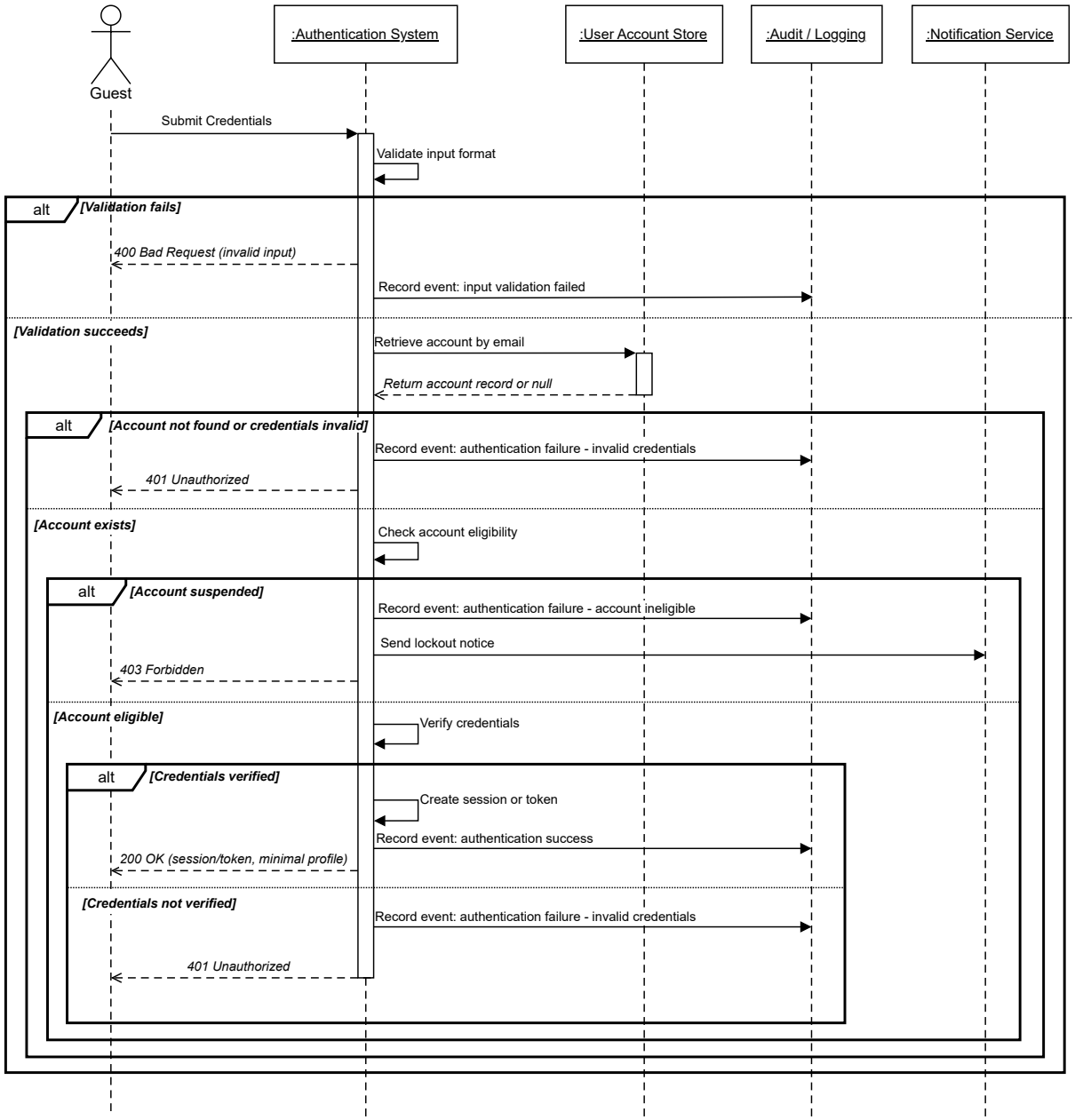


Figure 2.13: Sequence Diagram: User Authentication

Conclusion

This chapter established the conceptual foundation of the IoT-REAP platform across four complementary dimensions. The architectural analysis in Section 1 defined the five-layer MVC structure, justified the selection of the technology stack, and demonstrated alignment between architectural decisions and both functional and non-functional requirements. Section 2

modelled system behaviour through a general use case diagram and actor-specific refinements covering all four user roles. Section 3 provided a comprehensive class-level view of the domain model, grouping the thirty-one identified classes into five functional clusters. Finally, Section 4 captured the dynamic behaviour of critical workflows through sequence diagrams. Together, these artefacts form the design contract that guides the implementation and validation presented in the following chapter.

Chapter 3

Realization and Experimentation

Introduction

This chapter presents the practical realization of the IoT-REAP platform and provides evidence of its correct operation. It is organized into five main parts. Section 1 describes the hardware resources, development tools, and technology stack used during implementation. Section 2 presents the principal interfaces realized for each user role, from public visitors to platform administrators. Section 3 details the network topology and segmentation strategy validated through simulation. Section 4 covers the production deployment and security configuration. Finally, Section 5 reports the verification and testing activities conducted to validate functional correctness, role-based access control, and remote laboratory workflows.

1 Tools and Development Environment

The implementation of IoT-REAP required a development environment capable of supporting backend web application development, reactive frontend interfaces, virtual machine orchestration, and industrial remote access workflows. The working environment therefore combined hardware resources suitable for local development and testing with a modern software stack centered on Laravel, React, TypeScript, and MySQL [5, 8, 9, 15].

1.1 Hardware Environment

The hardware environment of IoT-REAP comprises the physical machines, network equipment, and tangible devices that provide the computational and operational foundation for remote laboratory work. Table 3.1 summarizes the hardware components and their respective roles within the platform.

Table 3.1: Physical Hardware Components of IoT-REAP

Hardware Category	**Main Components**	**Role in IoT-REAP**
Compute hosts	Proxmox VE server, dedicated VM host	Provides the physical compute, memory, and storage resources for hosting virtual machines. Session VMs use the dedicated VMID range 200–999 to avoid conflicts with reusable templates [2].

Hardware Category	Main Components	Role in IoT-REAP
Gateway node	Dedicated edge server or industrial PC	Acts as the physical bridge between the virtualized environment and external devices. This machine hosts the integration services that handle browser-based access, media streaming, camera handling, and peripheral management.
Camera hardware	USB webcams, IP cameras, ESP32-CAM devices, PTZ-capable cameras	Supplies live video sources for lab monitoring and experimentation. Default camera streams are configured at 640×480 resolution, 15 fps, and MJPEG input format.
USB peripherals	Cameras, serial adapters, and other attachable USB devices	Physical devices that can be bound to a running session when required, with tracked states: *available*, *bound*, *attached*, and *dedicated*.
Industrial devices	MQTT-connected robots, sensors, and automation hardware	Provides the physical interaction layer for robotics and industrial control scenarios [4].
Network infrastructure	Internal lab switches, routers, and cabling	Ensures reliable connectivity between compute hosts, gateway nodes, cameras, and industrial devices within the laboratory network.

This hardware structure reflects the layered physical architecture of the platform: compute hosts provide processing and storage resources, the gateway node enables connectivity between virtual and physical domains, camera and USB peripherals supply visual and interactive capabilities, and industrial devices deliver the hands-on experimentation layer required for remote laboratory work.

1.2 Development Environment

The development environment comprises the tools and utilities used to build, test, and document the IoT-REAP platform. Since the platform is implemented as a Laravel application using Inertia.js and React, both PHP and JavaScript tooling were required throughout the development cycle. Table 3.2 summarizes the main development tools and their roles.

Table 3.2: Development Environment

Tool	Logo	Role in the Project
Visual Studio Code		Used as the main integrated development environment for editing PHP, TypeScript, React components, route definitions, configuration files, and LaTeX report sources [19].
Git and GitHub		Used to manage source code history, feature evolution, and recovery of stable project states during iterative development [21, 22].
Composer		Used to install and manage Laravel framework packages and backend dependencies [23].
NPM		Used to manage frontend dependencies including React, TypeScript, Vite, Tailwind CSS, Radix UI components, Zustand, React Query, and Vitest [24].
Postman		Used to verify HTTP routes, request payloads, responses, authentication behavior, and external integration endpoints during development [25].
Draw.io		Used to prepare architecture, workflow, and UML diagrams integrated into the academic report [26].
OWASP ZAP		Used to perform security testing against the running application, including vulnerability scanning, authentication flow testing, and session management analysis [27].
Cisco Packet Tracer		Used to simulate the network topology and segmentation of the platform, demonstrating the isolation between corporate LAN, virtualization cluster, and external access points [28].

Tool	Logo	Role in the Project
VMware Workstation		Used to create and manage virtual machine templates for the Proxmox VE environment, including Windows and Linux images with preconfigured software for training sessions [29].

1.3 Technology Stack

The IoT-REAP platform relies on a diverse technology stack that supports application development, data management, remote laboratory operations, real-time communication, and multimedia services. Table 3.3 presents the main technologies organized by architectural layer.

Table 3.3: Technology Stack Used in the Platform

Technology	Logo	Role in the Platform
Backend and Application Layer		
PHP		Main backend programming language used to implement controllers, services, models, jobs, events, policies, and platform business logic [30].
Laravel		Backend framework used for routing, authentication, queue processing, validation, authorization, database access, and modular organization of application logic [5].
Inertia.js		Connects Laravel routes to React pages without building a separate REST-only frontend, enabling a smooth single-page navigation model [31].
Stripe		Supports checkout, payment tracking, refund workflows, and payout-related administrative features [7].
Frontend Layer		
React		Used to build the interactive frontend interfaces for training paths, dashboards, teaching pages, administration panels, and user settings [8].

Table 3.3 – Continued from previous page

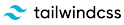
Technology	Logo	Role in the Platform
TypeScript		Provides type safety across the frontend codebase, improving maintainability and reducing interface integration errors [9].
Vite		Used for modern frontend asset bundling, hot module replacement in development, and production asset compilation [37].
Tailwind CSS		Used for responsive and utility-based user interface styling across public, learner, instructor, and administration pages [38].
Guacamole Common JS		Provides the JavaScript implementation of the Guacamole protocol for browser-based remote desktop access [3].
Data and Persistence Layer		
MySQL		Stores structured application data including users, reservations, certificates, quizzes, notifications, sessions, and payments [15].
Gateway and Streaming Services		
MediaMTX		Media server that handles RTSP, HLS, and WebRTC streaming for live camera feeds. Exposes ports 8554 (RTSP), 8888 (HLS), 8889 (WebRTC), and 9997 (API) [43, 44, 11, 45].
Frigate		Network video recorder that manages camera streams, motion detection, and recording for IP and USB cameras [46].
FFmpeg		Handles video transcoding across three quality profiles (360p, 720p, 1080p), HLS segment generation, thumbnail extraction, format normalization, and camera stream adaptation for browser-compatible playback [47, 11].
Apache Guacamole		Enables clientless browser-based remote desktop access to virtual machines without requiring local desktop client installation [3].

Table 3.3 – Continued from previous page

Technology	Logo	Role in the Platform
Device Integration and Messaging		
Mosquitto		MQTT broker that handles real-time robot command dispatch and telemetry reception using QoS 1 message delivery for reliable industrial control operations [49, 4].
Testing and Quality Assurance		
PHPUnit		Used to verify backend logic, route behavior, business rules, and Laravel feature flows [51].
Vitest		Used to test React interface logic and component behavior in the frontend codebase [52].

2 Implementation

This section presents the principal interfaces and functional areas implemented in IoT-REAP. The objective is not merely to present isolated screens, but to explain how the platform organizes the complete user experience across public access, technical learning, VM-based practice, teaching workflows, and administrative control. Interfaces are grouped according to the user role they primarily serve.

2.1 Landing Page

The landing page is the public entry point of the platform, implemented as a React frontend with TypeScript and styled with Tailwind CSS. It fetches featured training paths via Inertia.js shared page props from Laravel Eloquent queries, renders dynamic enrollment calls-to-action, and manages SEO metadata through Inertia's `Head` API, guiding visitors toward registration or login. A hero section communicates the platform's value proposition, while a curated catalog block presents popular training paths with their title, category, price, and ratings. The interface also exposes navigation toward the full catalog, individual path details, and authentication, providing a clear and frictionless onboarding path for new visitors.

[Screenshot to be inserted before final submission]

Landing page interface

Figure 3.1: Landing page interface

2.2 Authentication and Role Selection

The authentication interface manages the complete identity lifecycle of platform users. Registration collects essential profile information and triggers an email verification step through Laravel Fortify [32]. Login supports both credential-based and Google OAuth flows via Laravel Socialite [33, 34]. After successful authentication, users who have not yet committed to a role are presented with a role-selection screen allowing them to self-identify as a learner, teacher, or engineer. Two-factor authentication is available as an optional security layer via TOTP. This separation of roles from the earliest step of the user journey ensures that role-specific dashboards, permissions, and content are correctly applied from the first session.

[Screenshot to be inserted before final submission]

Authentication and role selection interface

Figure 3.2: Authentication and role selection interface

2.3 Learner Dashboard

The learner dashboard provides authenticated users with a consolidated view of their active learning context. It displays enrolled training paths alongside a visual progress indicator for each, recent activity entries, unread notifications, and quick access to earned or claimable certificates. A recommended content block surfaces paths related to the learner's existing enrollments, encouraging continued engagement. The dashboard is designed to reduce

navigation friction: a learner returning to the platform can resume any ongoing unit or launch a VM session without leaving this central hub. Data is fetched via TanStack React Query with Zustand managing lightweight client-side UI state [41].

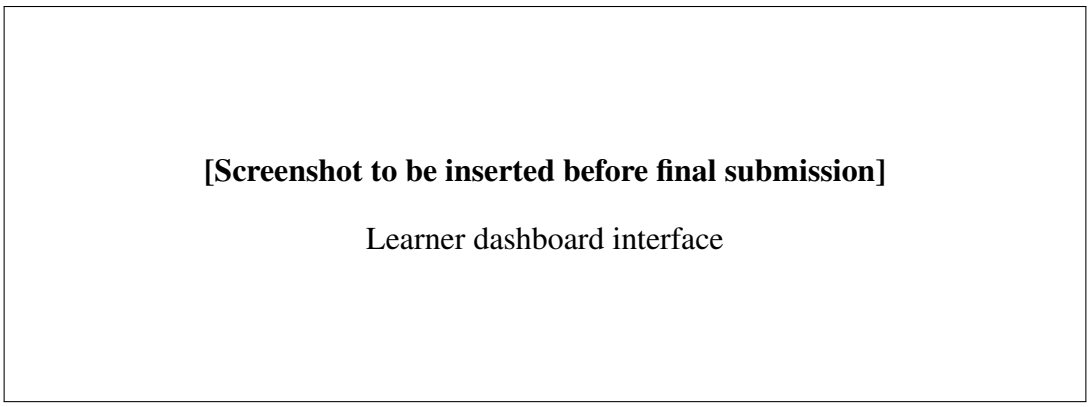


Figure 3.3: Learner dashboard interface

2.4 Training Path Catalog

The training path catalog allows authenticated and unauthenticated users to browse, search, and filter the full set of available learning content. Filters operate on category, price range, difficulty level, rating, and enrollment availability. Search is performed against path titles, descriptions, and instructor names. Each catalog card displays a thumbnail, title, instructor, rating summary, learner count, and price, giving users sufficient information to make enrollment decisions without entering the detail view. Pagination and sorting by relevance, popularity, or recency are supported.

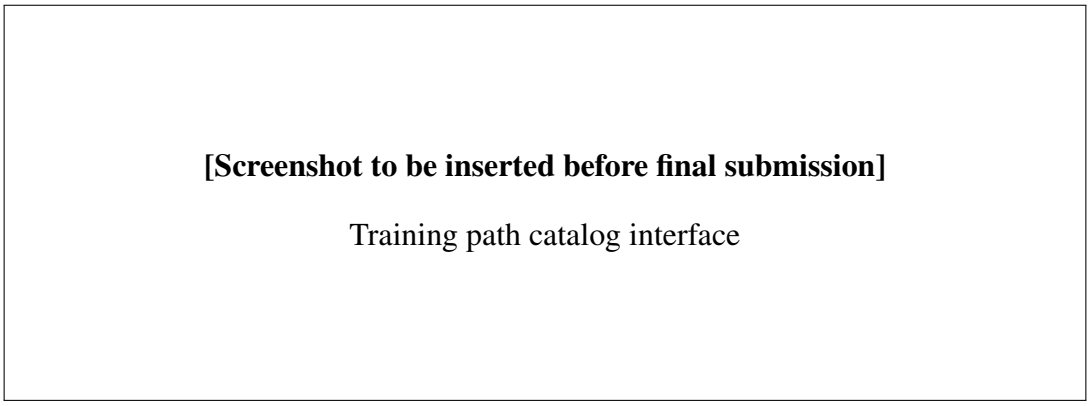


Figure 3.4: Training path catalog interface

2.5 Training Path Details

The training path details interface serves as the primary decision point before a learner purchases or enrolls in a path. It presents the full curriculum structure organized by unit,

the learning objectives, the instructor profile and credentials, aggregate ratings with individual review excerpts, and the enrollment or checkout action. For paths with associated VM sessions, the interface indicates the required laboratory access type and estimated session duration. Enrolled learners who revisit this page are redirected to their current unit position, avoiding redundant browsing.

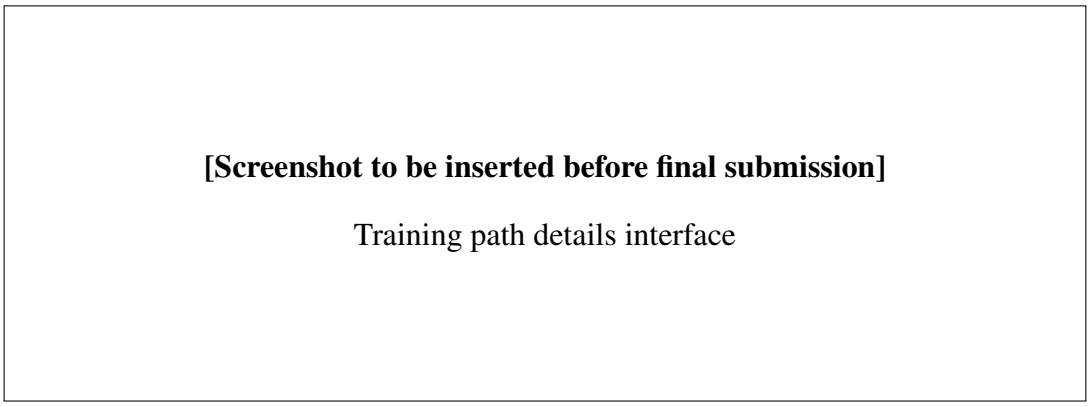


Figure 3.5: Training path details interface

2.6 Training Unit Learning Interface

The training unit interface is the core learning workspace of the platform. It combines a primary content area — displaying video content, article text, or embedded external resources — with a right-hand panel providing unit navigation, completion tracking, and discussion access. Learners can mark units as complete, take personal notes attached to specific content positions, and access the quiz associated with the unit when one is configured. For units linked to a practical VM session, a launch button is presented directly within the interface, allowing seamless transition from theory to hands-on practice without navigating away from the learning context.

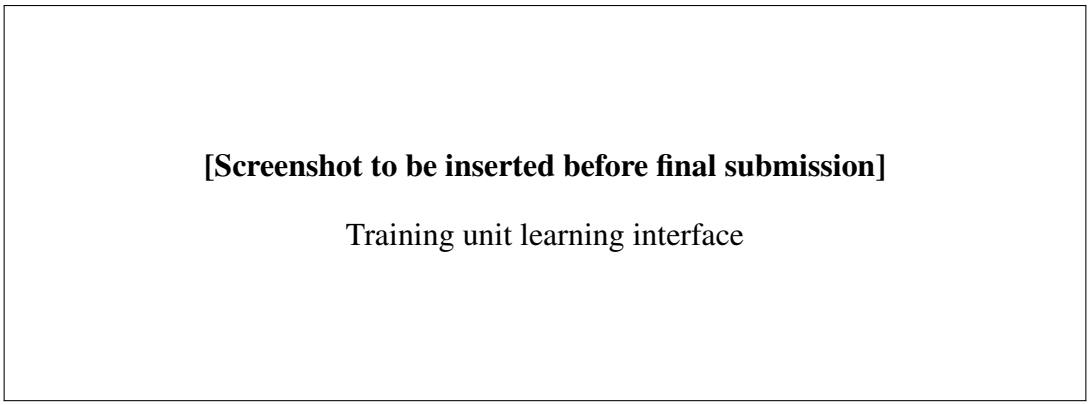


Figure 3.6: Training unit learning interface

2.7 Quiz Interface

The quiz interface supports structured knowledge evaluation within training paths. Questions are presented one at a time or in a paginated set, with support for multiple-choice and single-answer formats. The interface tracks the remaining time when a time limit is configured, and prevents submission until all mandatory questions have been answered. Upon submission, learners receive a results screen showing their score, correct answers, and per-question feedback. Multiple attempts are supported when enabled by the instructor, with attempt history accessible from the learner profile. Quiz completion contributes directly to unit progress and path completion status.

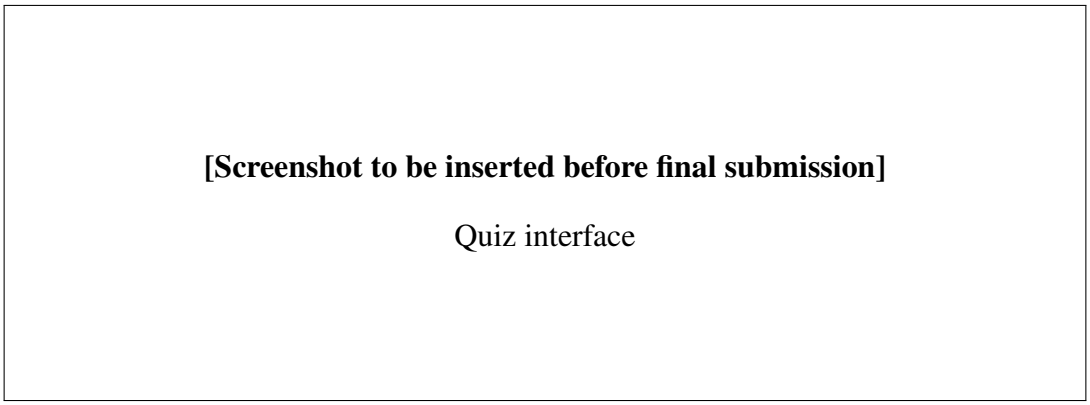


Figure 3.7: Quiz interface

2.8 Virtual Machine Session Interface

The virtual machine session interface gives learners and engineers browser-based access to a provisioned remote laboratory environment. It displays the current VM state, connection status, and session time remaining, and embeds the Apache Guacamole client via Guacamole Common JS to establish a clientless remote desktop tunnel directly within the page [3]. Controls allow the user to extend the session within authorized limits, request a full-screen view, and terminate the session explicitly. If a session is lost due to a connectivity interruption, the interface provides a reconnect action without requiring a new reservation. VM-specific metadata — operating system, assigned resources, and associated training unit — is displayed in a sidebar to maintain context during the practical activity.

2.9 Session Hardware and Camera Panels

The session hardware and camera panels extend the VM session interface with access to the physical laboratory resources assigned to the active session. The camera panel renders live video streams from webcams, IP cameras, or ESP32-CAM devices via WebRTC or HLS,

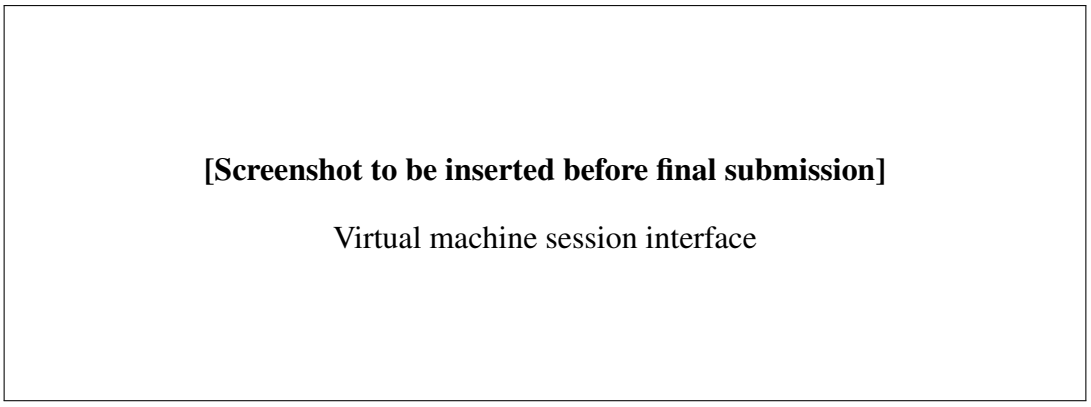


Figure 3.8: Virtual machine session interface

depending on the source configuration [45, 11]. The hardware panel exposes USB devices in their current binding state (*available*, *bound*, *attached*, or *dedicated*) and provides attach and detach controls when the session is authorized for peripheral access. Industrial device controls, when applicable, expose MQTT-based command dispatch for robots and sensors connected to the lab network [4].

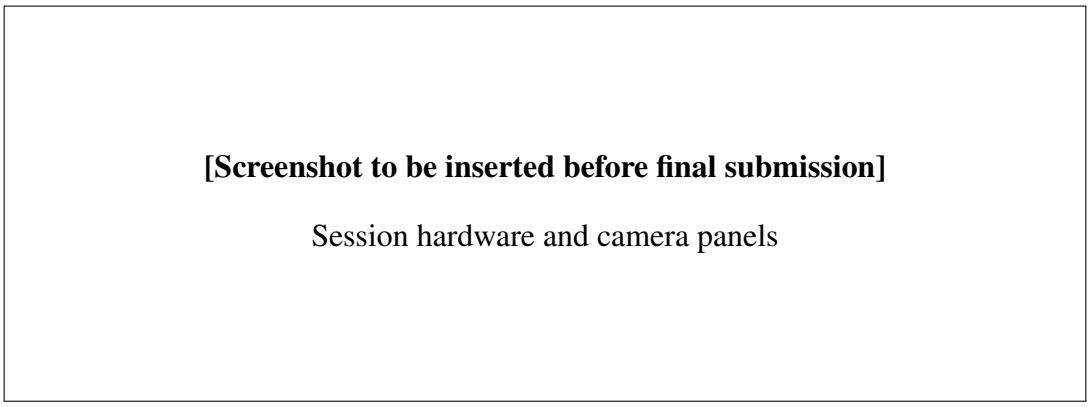


Figure 3.9: Session hardware and camera panels

2.10 Reservations Interface

The reservations interface allows users to plan and manage scheduled access to laboratory resources. Available time slots are displayed in a calendar-based view, with resource availability derived from the current session schedule and administrator-defined maintenance windows. Users can submit reservation requests specifying the desired resource type, duration, and preferred time range. The interface displays the real-time status of submitted reservations (*pending*, *confirmed*, *active*, *completed*, or *cancelled*), and notifies users of status changes through the platform notification system.

[Screenshot to be inserted before final submission]

Reservations interface

Figure 3.10: Reservations interface

2.11 Certificates Interface

The certificates interface allows learners to consult, claim, and download completion certificates generated upon finishing an eligible training path. Each certificate is identified by a unique, publicly verifiable UUID that can be checked by third parties through a dedicated public URL, without requiring authentication. The interface displays the learner's name, the path title, the completion date, and the issuing organization. Downloaded certificates are rendered as PDF documents. Administrators can configure certificate templates and define the completion threshold required for eligibility at the platform level.

[Screenshot to be inserted before final submission]

Certificates interface

Figure 3.11: Certificates interface

2.12 Payment and Refund Interface

The payment and refund interface manages the financial interactions between learners and the platform. Checkout is handled through Stripe Elements, with the interface displaying order summary, price breakdown, and available payment methods before confirming the transaction [7]. After checkout, learners receive a confirmation and can access their payment history, which records all transactions with their date, amount, and status. Refund requests can be submitted within the eligible window and are tracked through a status workflow (*requested*, *under review*,

approved, *rejected*). The interface surfaces refund outcomes directly alongside the relevant payment record. Stripe webhook events are verified server-side through signature validation to ensure asynchronous payment state integrity.

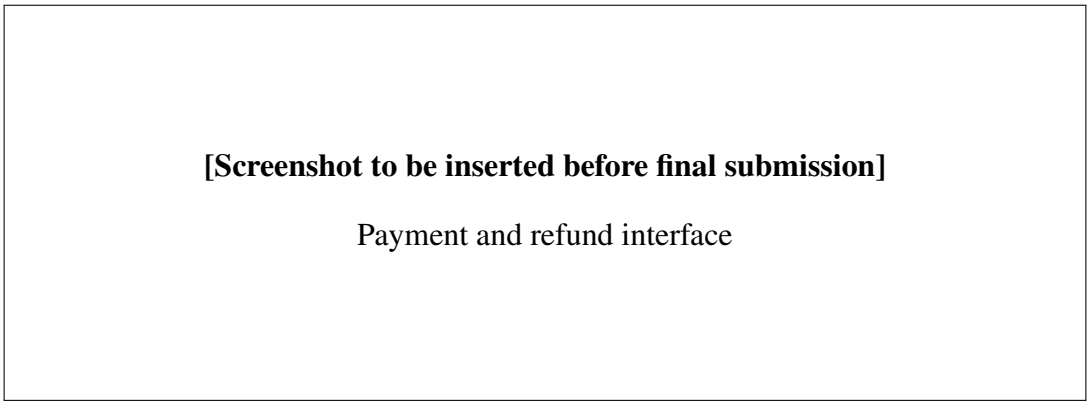


Figure 3.12: Payment and refund interface

2.13 Teacher Content Studio

The teacher content studio is the primary authoring environment for approved instructors. It allows teachers to create and structure training paths by defining metadata, organizing units in a drag-and-drop sequence, and attaching content items — videos, articles, and quizzes — to each unit. Video uploads are processed server-side through FFmpeg transcoding into multiple quality profiles [47]. Each unit can be linked to a VM session type, defining the practical environment that learners will access. The studio also exposes submission and revision workflows, through which teaching content is proposed for administrator approval before becoming publicly accessible.

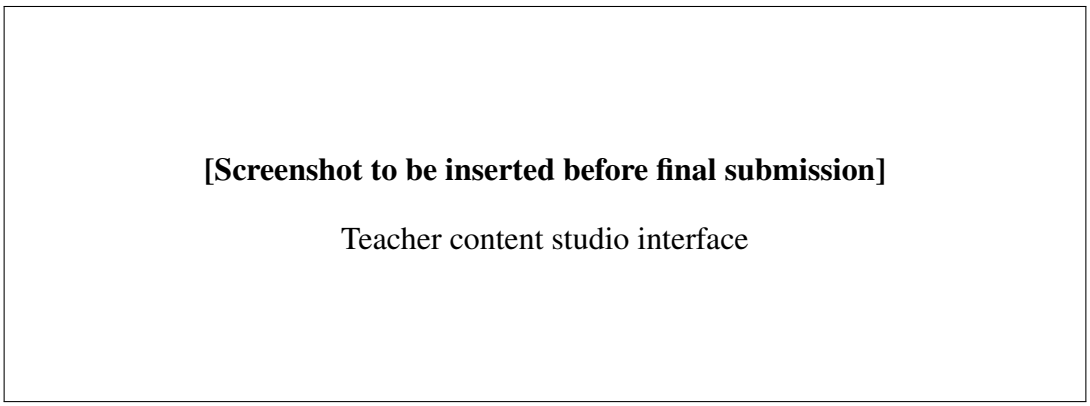


Figure 3.13: Teacher content studio interface

2.14 Teacher Analytics and Earnings

The teacher analytics and earnings interface presents instructors with a quantitative view of their content's performance. Key indicators include total enrollments per path, active learner count, average quiz scores, unit completion rates, and overall training path rating. Revenue evolution is rendered as a time-series chart using Recharts [42], with breakdowns by path and billing period. Teachers can submit payout requests when their accumulated balance exceeds the minimum threshold, and track the status of submitted payout requests from a financial summary panel.

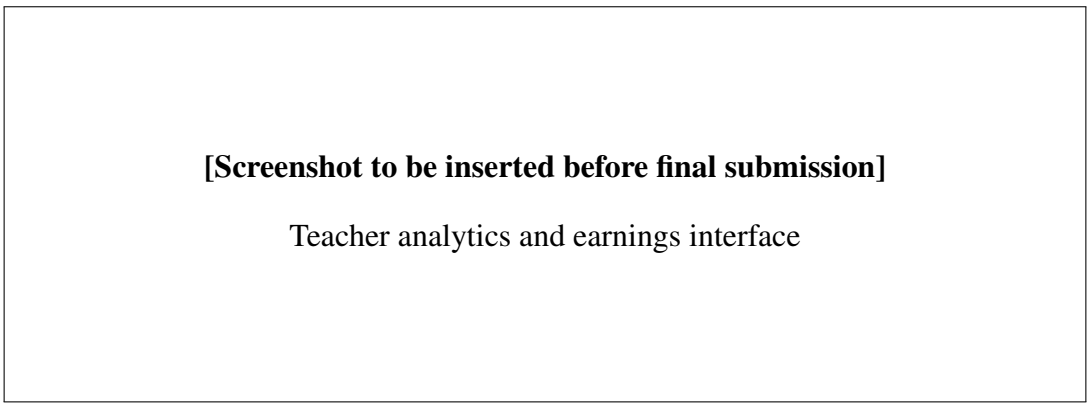


Figure 3.14: Teacher analytics and earnings interface

2.15 Administration Dashboard

The administration dashboard provides platform operators with centralized control over all operational dimensions of IoT-REAP. It organizes management functions into dedicated panels: user management (creation, role assignment, suspension), training path review and approval, reservation supervision, infrastructure monitoring (VM pool status, gateway connectivity, camera availability), financial oversight (payments, refunds, payout processing), and alert management (system warnings, maintenance scheduling, moderation flags). The dashboard surfaces real-time indicators through WebSocket-delivered notifications powered by Laravel Reverb [35], enabling administrators to respond to operational events without manual page refresh.

3 Network Topology and Segmentation

To illustrate the practical implementation of the proposed remote access architecture, a network simulation was conducted using Cisco Packet Tracer [28]. The simulated topology reflects a real-world scenario in which strict network segmentation is enforced between the corporate LAN, the virtualization cluster, and external access points.

[Screenshot to be inserted before final submission]

Administration dashboard interface

Figure 3.15: Administration dashboard interface

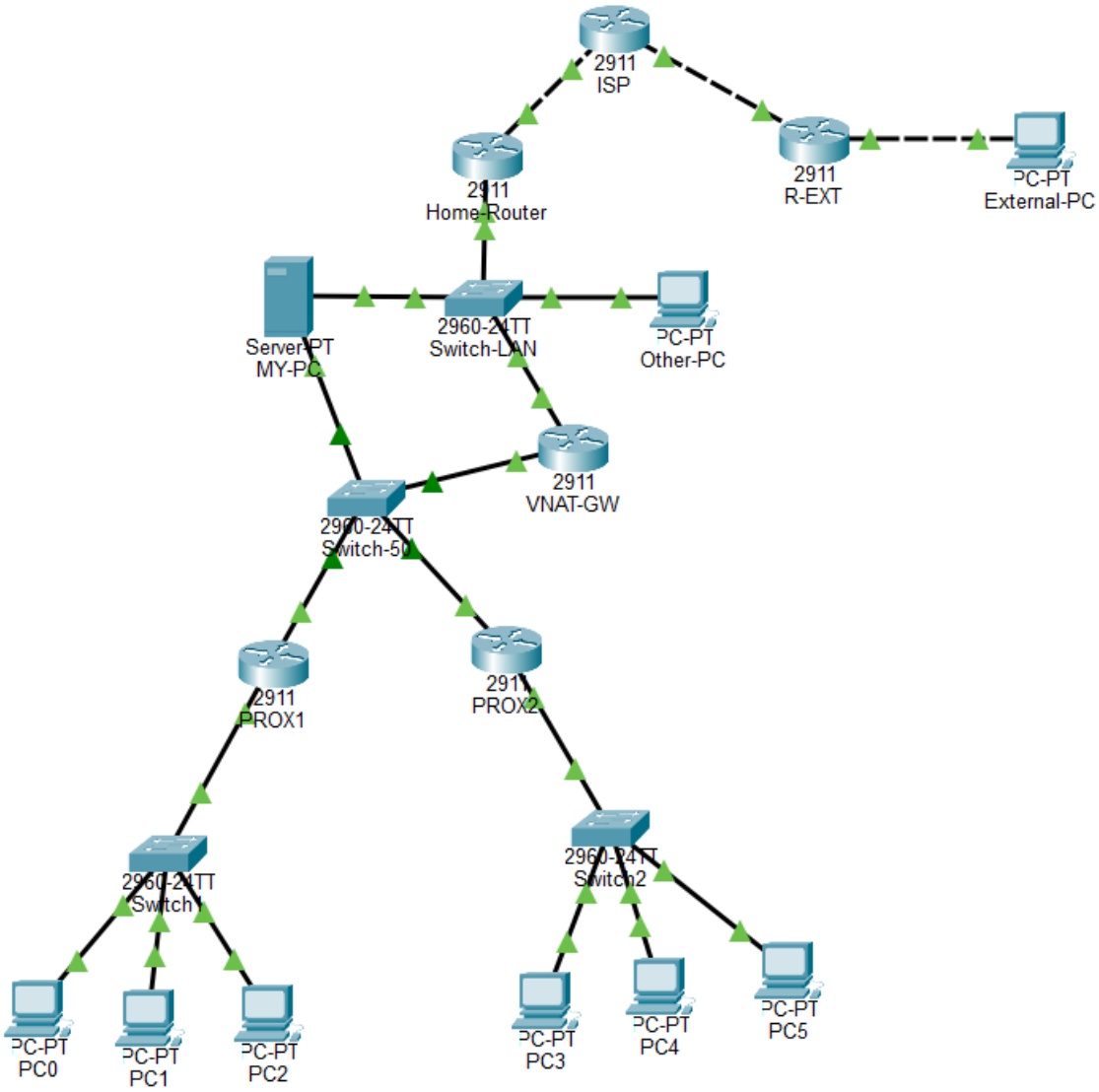


Figure 3.16: Simulated Network Topology — Cisco Packet Tracer

The topology is structured into four distinct network segments, each serving a specific security and operational role:

- **Home/Corporate LAN (192.168.1.0/24):** Represents the end-user environment. Only authorized devices within this segment are permitted to interact with the virtualization layer. Unauthorized devices such as general workstations are explicitly blocked from accessing internal cluster networks through access control lists enforced at the gateway level.
- **VMware NAT Gateway (VNAT-GW):** Acts as the sole entry point between the corporate LAN and the Proxmox cluster network. This device enforces Network Address Translation (NAT) and routing policies, ensuring that the internal cluster addressing scheme (192.168.50.0/24) remains hidden from the broader network, consistent with the principle of least exposure.
- **Proxmox Cluster Network (192.168.50.0/24):** Hosts two Proxmox virtualization servers (PROX1 and PROX2), each managing an isolated VM subnet (192.168.51.0/24 and 192.168.52.0/24 respectively). This segment is entirely unreachable from the public internet and is only accessible through the VNAT-GW, enforcing strict east-west traffic control within the OT-adjacent environment.
- **External VPN Access:** Remote access is simulated through a dedicated external router (R-EXT) connected via an ISP segment. A VPN tunnel (10.10.10.0/30) is established between the external endpoint and the internal gateway, replicating the behavior of a zero-trust network access tool such as Netbird. This ensures that external clients never directly expose or access internal subnet addresses, and all traffic is encapsulated within the authenticated tunnel.

This segmented architecture directly addresses the security concerns outlined in the previous chapter. By enforcing layer-3 isolation between network zones, applying NAT to conceal internal addressing, and restricting external access exclusively through an encrypted VPN tunnel, the design adheres to the core principles of the Purdue model. No direct path exists between the public internet and the virtualization cluster, and lateral movement between network segments is controlled entirely through explicit routing policies and access control lists.

4 Hosting

Having defined the network architecture for lab access, this section describes how the platform itself is deployed on a production cloud infrastructure. The IoT-REAP platform is

deployed on OVH cloud infrastructure at `iotreap.novationcityapp.com`, providing production-grade hosting with enterprise-level reliability and secure HTTPS access through Let's Encrypt SSL/TLS certificates with automatic renewal [53, 54].

4.1 OVH Hosting Infrastructure

OVH delivers robust infrastructure supporting the platform's operational requirements through a Virtual Private Server (VPS) solution with a dedicated multi-core CPU, sufficient RAM for concurrent sessions, and SSD storage ensuring optimal database performance.

The server runs Ubuntu 22.04 LTS for long-term stability. Nginx serves as the web server and reverse proxy with optimized Laravel settings including gzip compression, browser caching, and rate limiting. PHP 8.4 with PHP-FPM handles execution using OPcache bytecode caching and optimized memory limits [55, 56, 30].

MySQL 8.4 operates with production-tuned configurations including InnoDB buffer pool optimization and proper indexing, delivering sub-50 ms average query response times. Automated daily backups provide disaster recovery capability and guarantee data integrity [15].

4.2 Domain Configuration and SSL Security

The domain `iotreap.novationcityapp.com` resolves to the OVH server via a DNS A record. Let's Encrypt certificates provide HTTPS encryption for all client-server communications, with Certbot handling automatic renewal before certificate expiry [54, 57].

Nginx is configured to enforce HTTPS redirection, automatically upgrading all HTTP requests to secure connections. Security headers including HSTS (HTTP Strict Transport Security), `X-Frame-Options`, and Content Security Policy (CSP) are applied at the reverse proxy level, protecting against common web vulnerabilities such as clickjacking and cross-site scripting (XSS) attacks [56, 58, 59].

5 Tests and Verification

The verification of IoT-REAP was conducted across four main categories: functional correctness, access control, remote laboratory workflows, and performance. The objective was to confirm that implemented interfaces, authorization rules, laboratory services, and learning features behaved correctly and consistently, both in isolation and as an integrated system.

5.1 Functional Verification

Functional verification covered the primary user journeys for each role, from public navigation through to certification. Table 3.4 presents a representative selection of the test scenarios executed, their preconditions, expected outcomes, and observed results.

Table 3.4: Representative Functional Test Cases

Scenario	Preconditions	Expected Result	Observed Result	Status
Learner registration with email verification	No existing account	Account created; verification email sent; login blocked until verified	Account created; email received within 5 s; login blocked as expected	**Pass**
Google OAuth login	Valid Google account	User authenticated; redirected to role selection or dashboard	Authenticated; redirected to `/dashboard` within 800 ms	**Pass**
Enrollment in a paid path	Authenticated learner; Stripe test mode active	Payment processed; enrollment created; unit access granted	Stripe Checkout completed; enrollment persisted; access granted immediately	**Pass**
Unit completion and progress update	Learner enrolled; unit not yet completed	Completion state persisted; path progress percentage updated	Progress updated from 40% to 60% after marking unit complete	**Pass**
Quiz submission and result display	Learner enrolled; quiz attached to unit	Score computed; correct answers displayed; attempt recorded	Score 8/10 displayed; attempt logged in history	**Pass**
VM session creation and browser access	Active reservation; Proxmox template available	VM provisioned in VMID range 200–999; Guacamole session accessible	VM created with VMID 205; accessible via browser within 35 s	**Pass**
Camera stream display in session	Gateway online; camera attached to session	WebRTC or HLS stream rendered in hardware panel	Stream rendered within 3 s of panel open	**Pass**

Scenario	Preconditions	Expected Result	Observed Result	Status
Certificate generation on path completion	All required units and quizzes completed	Certificate created with unique UUID; downloadable as PDF	Certificate generated; verified via public URL	**Pass**
Teacher submits path for approval	Teacher role; path draft complete	Path enters pending state; administrator notified	Pending status set; notification delivered via Reverb WebSocket channel	**Pass**
Administrator approves a training path	Path in pending state	Path published; teacher notified	Path visible in catalog; teacher notified via dashboard alert	**Pass**

5.2 Access Control Verification

Authorization was verified to confirm that each role could access only the interfaces and actions assigned to it. Learners attempting to access teacher or administration routes received HTTP 403 Forbidden responses. Teachers were blocked from administrative functions such as user management and infrastructure controls. Engineers had access to VM session provisioning and hardware management, but not to course authoring or financial administration. These boundaries were enforced at the Laravel policy layer and validated through both manual browser testing and PHPUnit feature tests targeting route middleware [51].

5.3 Remote Laboratory Workflow Verification

The remote laboratory workflow was verified through the full lifecycle of a VM session: creation from a Proxmox template, browser-based access via Guacamole, session extension, hardware panel interaction (camera stream and USB device binding), and explicit session termination [3]. Gateway availability, device assignment state transitions, and session cleanup after termination were confirmed to behave consistently across multiple test cycles.

5.4 Performance Observations

Table 3.5 summarizes key performance indicators observed during testing on the production OVH VPS environment. These figures are indicative rather than statistically rigorous and are intended to validate that the platform meets basic operational usability thresholds.

Table 3.5: Key Performance Observations (Production Environment)

Metric	Observed Value
Average MySQL query response time	< 50 ms
VM session startup time (template clone to Guacamole access)	25–45 s
Camera stream first-frame latency (WebRTC)	< 3 s
Dashboard page load time (cold cache)	< 1.5 s
Certificate PDF generation time	< 2 s

5.5 Automated Test Coverage

Automated testing complemented manual verification across both application layers. Backend behavior was validated with PHPUnit, covering Laravel route authorization, business rule enforcement (reservation conflict detection, VMID range allocation, quiz attempt limits), and feature-level integration tests for authentication and payment workflows [51]. Frontend component behavior was validated with Vitest for selected React components, focusing on form validation logic, state transitions in the session interface, and conditional rendering based on user role [52]. Together, these automated tests provided a repeatable regression baseline throughout the iterative development process.

Conclusion

This chapter presented the complete technical realization of IoT-REAP, from the hardware and software environment through to production deployment and verification. The development stack — centered on Laravel, React, TypeScript, and MySQL — proved well-suited to the platform's requirements, providing a coherent architecture for user management, content delivery, remote laboratory access, and real-time communication. The full set of implemented interfaces covers the complete range of user roles, and the network topology simulation demonstrated that the proposed segmentation strategy is practically enforceable. Verification activities confirmed that the platform satisfies its core functional requirements: learners can progress through structured content and access physical laboratory resources from the browser; teachers can author and monitor their content; and administrators retain full operational control. The remaining limitations — principally the absence of a statistically rigorous load test and the reliance on manual performance observation — constitute areas for future improvement and are discussed in the general conclusion.

General Conclusion

As part of our **(four-month)** internship at Novation City, we designed and implemented IoT-REAP, a secure industrial remote operations and training platform for Industry 4.0 learning and experimentation. The platform delivers six integrated functional domains — session management, VM provisioning, remote desktop access, camera supervision, IoT device control, and structured training management — under a unified, role-differentiated interface serving Engineer, Teacher, and Administrator profiles. The objective was to eliminate dependency on physical laboratory presence by providing a coherent environment in which users can launch sessions, interact with real equipment, follow guided learning paths, and monitor activity from a single web interface.

Our development approach was iterative and architecture-driven, following an Agile-inspired process with short delivery cycles. On the backend, Laravel was organized around FormRequest validation, thin controllers, service-layer business logic, and repository-based data access. On the frontend, React with TypeScript provided typed API integrations and reusable component structures that improved maintainability as project scope expanded. Role-based access flows ensure that each profile encounters only contextually relevant actions and data, forming a solid foundation for integrating complex external systems [5, 8, 9].

A major contribution was the implementation of the remote lab session lifecycle. Proxmox integration enables on-demand VM provisioning and lifecycle control, while Guacamole-based browser access allows users to open RDP, VNC, or SSH sessions without local client installation. Tokenized access patterns, expiration-aware session handling, and resource cleanup mechanisms prevent orphaned VMs and maintain platform stability, establishing a bridge between cloud orchestration and practical industrial training [2, 3].

A second major contribution was the integration of real-time hardware context into active training sessions. Camera services and gateway integrations allow users to observe physical equipment remotely, while PTZ control is routed through MQTT-driven command channels in a structured and auditable manner. USB device reservation and assignment workflows enable specific hardware to be attached to live sessions in a controlled way, positioning IoT-REAP as an operational remote laboratory rather than a conventional e-learning portal [4, 50].

Beyond infrastructure access, the platform supports instructor-driven training path management. Teachers can define learning sequences linked to specific VM sessions, ensuring that pedagogical structure and practical execution remain synchronized within a single workflow. Learners progress through guided material while interacting directly with realistic industrial environments, developing operational competencies through practical engagement rather than simulated abstractions.

Delivering these capabilities, however, required navigating several significant engineering challenges. The first was coordinating heterogeneous technologies with fundamentally different operational behaviors: Proxmox operations are asynchronous, remote desktop tunnels are state-sensitive, MQTT control requires consistent topic discipline, and camera streaming introduces network variability. We addressed this by isolating external provider logic in dedicated service classes, standardizing integration contracts, and improving observability through explicit logging and defensive checks.

The second challenge was preserving architectural quality as feature scope expanded rapidly. Keeping controllers thin, preventing business logic from leaking across layers, and maintaining typed frontend contracts required discipline enforced through coding standards, incremental API documentation, and targeted refactoring. This reinforced a key engineering lesson: in production platforms, maintainability is a first-class deliverable, not a post-project cleanup activity.

The third challenge was balancing delivery velocity with security and reliability requirements. Because IoT-REAP handles privileged operations — VM control, remote access, and hardware assignment — authorization boundaries had to be explicit and verifiable. We reinforced input validation and role scoping on sensitive endpoints, tightened resource ownership checks, and introduced regression tests on security-critical paths. Combined with static type checking and both unit and feature testing, this produced a higher-assurance, auditable development workflow.

Looking forward, several directions can extend the platform's capabilities. Completing high-impact API integrations and advancing frontend parity for administrative and teaching workflows represents the immediate priority. Deeper analytics on session usage, infrastructure efficiency, and learner progression would support data-driven operational decisions. Predictive scheduling and optimized resource allocation would reduce reservation wait times, while enhanced zero-trust controls, compliance-oriented audit reporting, and mobile-optimized interfaces would broaden operational reach.

This internship produced tangible technical outcomes while developing engineering competencies across full-stack architecture, distributed system integration, and security-aware design. Isolating Proxmox asynchrony in service classes, enforcing repository patterns to preserve testability, and managing MQTT topic discipline under strict authorization constraints represented concrete engineering decisions that extended and deepened the architectural foundations of our academic training.

IoT-REAP delivers a technically sound and extensible foundation connecting virtual infrastructure, remote hardware interaction, and structured learning within a single platform. Core functional requirements were verified through unit and feature tests, confirming alignment with the initial specifications. The system demonstrates the feasibility of secure, remote

Industry 4.0 training at scale, and its architecture is designed to accommodate a growing operational roadmap.

Webgraphy

[1] Novation City, "Novation City," *Novation City*, https://novationcity.com, Accessed: April 23, 2026.

[2] Proxmox Server Solutions GmbH, "Proxmox VE API," *Proxmox VE Wiki*, https://pve.proxmox.com/wiki/Proxmox_VE_API, Accessed: February 15, 2026.

[3] The Apache Software Foundation, "Apache Guacamole Manual," *Apache Guacamole*, https://guacamole.apache.org/doc/gug/, Accessed: February 4, 2026.

[4] OASIS Open, "MQTT Version 5.0," *OASIS Open*, https://www.oasis-open.org/standard/mqtt-v5-0/, Accessed: May 6, 2026.

[5] Laravel LLC, "Laravel 12.x Documentation," *Laravel*, https://laravel.com/docs/12.x, Accessed: March 4, 2026.

[6] Sebastian Ramirez, "FastAPI Documentation," *FastAPI*, https://fastapi.tiangolo.com/, Accessed: March 1, 2026.

[7] Stripe, Inc., "Stripe Documentation," *Stripe Docs*, https://docs.stripe.com/, Accessed: February 18, 2026.

[8] Meta Open Source, "React Documentation," *React*, https://react.dev/, Accessed: May 6, 2026.

[9] Microsoft Corporation, "TypeScript Documentation," *TypeScript*, https://www.typescriptlang.org/docs/, Accessed: February 14, 2026.

[10] pmndrs, "Zustand Documentation," *Zustand*, https://zustand.docs.pmnd.rs/, Accessed: April 28, 2026.

[11] RFC Editor, "RFC 8216: HTTP Live Streaming," *RFC Editor*, https://www.rfc-editor.org/rfc/rfc8216, Accessed: May 6, 2026.

[12] OWASP Foundation, "Zed Attack Proxy Documentation," *OWASP ZAP*, https://www.zaproxy.org/docs/, Accessed: May 26, 2026.

[13] Grady Booch, James Rumbaugh, and Ivar Jacobson, "The Unified Modeling Language User Guide, 2nd ed.," *Addison-Wesley*, https://personal.utdallas.edu/~chung/Fujitsu/UML_2.0/Rumbaugh--UML_2.0_Reference_CD.pdf, Accessed: April 11, 2026.

[14] Scott W. Ambler, "Agile Documentation," *Agile Modeling*, https://agilemodeling.com/essays/agiledocumentation.htm, Accessed: February 12, 2026.

[15] Oracle Corporation, "MySQL 8.4 Reference Manual," *MySQL Documentation*, https://dev.mysql.com/doc/refman/8.4/en/, Accessed: April 17, 2026.

[16] Redis Ltd., "Redis Documentation," *Redis Docs*, https://redis.io/docs/latest/, Accessed: March 27, 2026.

[17] Docker, Inc., "Docker Documentation," *Docker Docs*, https://docs.docker.com/, Accessed: February 5, 2026.

[18] Martin Fowler, "Patterns of Enterprise Application Architecture," *Enterprise Application Architecture*, https://dl.ebooksworld.ir/motoman/Patterns%20of%20Enterprise%20Application%20Architecture.pdf, Accessed: February 4, 2026.

[19] Microsoft Corporation, "Visual Studio Code Documentation," *Visual Studio Code*, https://code.visualstudio.com/docs, Accessed: February 12, 2026.

[20] Laravel LLC, "Laravel Herd Documentation," *Laravel Herd*, https://herd.laravel.com/docs/windows/getting-started/installation, Accessed: February 28, 2026.

[21] Software Freedom Conservancy, "Git Documentation," *Git*, https://git-scm.com/docs, Accessed: March 2, 2026.

[22] GitHub, Inc., "GitHub Docs," *GitHub Docs*, https://docs.github.com/en, Accessed: April 6, 2026.

[23] Nils Adermann and Jordi Boggiano, "Composer Documentation," *Composer*, https://getcomposer.org/doc/, Accessed: April 19, 2026.

[24] npm, Inc., "npm Docs," *npm Docs*, https://docs.npmjs.com/, Accessed: February 4, 2026.

[25] Postman, Inc., "Postman Docs," *Postman Learning Center*, https://learning.postman.com/, Accessed: April 13, 2026.

[26] JGraph Ltd., "diagrams.net," *diagrams.net*, https://www.diagrams.net/, Accessed: February 26, 2026.

[27] OWASP Foundation, "Zed Attack Proxy Documentation," *OWASP ZAP*, https://www.zaproxy.org/docs/, Accessed: June 3, 2026.

[28] Cisco Systems, Inc., "Cisco Packet Tracer Documentation," *Cisco Networking Academy*, https://www.netacad.com/cisco-packet-tracer, Accessed: May 12, 2026.

[29] Broadcom Inc., "VMware Technical Documentation," *Broadcom TechDocs*, https://techdocs.broadcom.com/, Accessed: April 3, 2026.

[30] The PHP Group, "PHP Manual," *PHP Documentation*, https://www.php.net/docs.php, Accessed: May 3, 2026.

[31] Inertia.js, "Inertia.js Documentation," *Inertia.js*, https://inertiajs.com/, Accessed: April 25, 2026.

[32] Laravel LLC, "Laravel Fortify Documentation," *Laravel*, https://laravel.com/docs/12.x/fortify, Accessed: May 1, 2026.

[33] Laravel LLC, "Laravel Socialite Documentation," *Laravel*, https://laravel.com/docs/12.x/socialite, Accessed: April 11, 2026.

[34] Google, "Using OAuth 2.0 to Access Google APIs," *Google Identity*, https://developers.google.com/identity/protocols/oauth2, Accessed: March 26, 2026.

[35] Laravel LLC, "Laravel Reverb Documentation," *Laravel*, https://laravel.com/docs/12.x/reverb, Accessed: March 1, 2026.

[36] Laravel LLC, "Laravel Telescope Documentation," *Laravel*, https://laravel.com/docs/12.x/telescope, Accessed: March 30, 2026.

[37] Vite, "Vite Documentation," *Vite*, https://vite.dev/guide/, Accessed: April 17, 2026.

[38] Tailwind Labs, "Tailwind CSS Documentation," *Tailwind CSS*, https://tailwindcss.com/docs, Accessed: March 8, 2026.

[39] WorkOS, "Radix UI Documentation," *Radix UI*, https://www.radix-ui.com/primitives/docs/overview/introduction, Accessed: May 15, 2026.

[40] React Hook Form, "React Hook Form Documentation," *React Hook Form*, https://react-hook-form.com/docs, Accessed: May 23, 2026.

[41] TanStack, "TanStack Query Documentation," *TanStack*, https://tanstack.com/query/latest/docs/framework/react/overview, Accessed: February 1, 2026.

[42] Recharts Group, "Recharts Documentation," *Recharts*, https://recharts.org/en-US/, Accessed: May 9, 2026.

[43] bluenviron, "MediaMTX," *GitHub*, https://github.com/bluenviron/mediamtx, Accessed: May 15, 2026.

[44] RFC Editor, "RFC 2326: Real Time Streaming Protocol," *RFC Editor*, https://www.rfc-editor.org/rfc/rfc2326, Accessed: February 21, 2026.

[45] World Wide Web Consortium, "WebRTC: Real-Time Communication in Browsers," *W3C Recommendation*, https://www.w3.org/TR/webrtc/, Accessed: May 1, 2026.

[46] Frigate NVR, "Frigate Documentation," *Frigate*, https://docs.frigate.video/, Accessed: March 27, 2026.

[47] FFmpeg, "FFmpeg Documentation," *FFmpeg*, https://ffmpeg.org/documentation.html, Accessed: March 16, 2026.

[48] IETF, "WebRTC-HTTP Egress Protocol (WHEP)," *IETF Datatracker*, https://datatracker.ietf.org/doc/draft-ietf-wish-whep/, Accessed: March 8, 2026.

[49] Eclipse Foundation, "Eclipse Mosquitto Documentation," *Eclipse Mosquitto*, https://mosquitto.org/documentation/, Accessed: February 20, 2026.

[50] The Linux Kernel Organization, "USB/IP Protocol," *Linux Kernel Documentation*, https://docs.kernel.org/usb/usbip_protocol.html, Accessed: February 28, 2026.

[51] Sebastian Bergmann, "PHPUnit Documentation," *PHPUnit*, https://docs.phpunit.de/, Accessed: May 9, 2026.

[52] Vitest, "Vitest Documentation," *Vitest*, https://vitest.dev/guide/, Accessed: March 16, 2026.

[53] OVHcloud, "VPS," *OVHcloud*, https://www.ovhcloud.com/en/vps/, Accessed: February 14, 2026.

[54] Internet Security Research Group, "Let's Encrypt Documentation," *Let's Encrypt*, https://letsencrypt.org/docs/, Accessed: February 12, 2026.

[55] Canonical Ltd., "Ubuntu Documentation," *Ubuntu Help*, https://help.ubuntu.com/, Accessed: March 21, 2026.

[56] F5, Inc., "nginx Documentation," *nginx*, https://nginx.org/en/docs/, Accessed: February 13, 2026.

[57] Electronic Frontier Foundation, "Certbot Documentation," *Certbot*, https://eff-certbot.readthedocs.io/en/stable/, Accessed: March 18, 2026.

[58] Mozilla, "Strict-Transport-Security," *MDN Web Docs*, `https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Headers/Strict-Transport-Security`, Accessed: May 20, 2026.

[59] Mozilla, "Content Security Policy (CSP)," *MDN Web Docs*, `https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/CSP`, Accessed: March 17, 2026.

Abstract

This report presents IoT-REAP, a secure industrial remote operations platform for Industry 4.0. The platform addresses challenges in remote access, such as VPN complexity, session management, and OT/IT network isolation.

Developed with Laravel, React, TypeScript, and Python FastAPI, IoT-REAP integrates Proxmox VE for VM provisioning, Apache Guacamole for browser-based remote desktop, and MQTT for IoT robot control. Innovations include zero-trust security, device fingerprinting, geo-IP anomaly detection, real-time risk scoring, AI-powered demand forecasting, and predictive maintenance.

The system supports 50+ concurrent users with automatic VM provisioning, sub-200ms robot telemetry latency, and tamper-evident audit logging with security-focused access controls. Results show secure, browser-based access to industrial systems with no client installation.

Keywords: Industry 4.0, IoT, Remote Access, Proxmox VE, Apache Guacamole, MQTT, Zero-Trust Security, Predictive Maintenance, Machine Learning

Résumé

Ce rapport présente IoT-REAP, une plateforme sécurisée d'opérations industrielles à distance pour l'Industrie 4.0. Elle répond aux défis de l'accès distant, comme la complexité des VPN, la gestion des sessions et l'isolation des réseaux OT/IT.

Développée avec Laravel, React, TypeScript et Python FastAPI, IoT-REAP intègre Proxmox VE pour le provisionnement de VM, Apache Guacamole pour l'accès bureau à distance, et MQTT pour le contrôle de robots IoT. Les innovations incluent la sécurité zero-trust, empreinte d'appareil, détection d'anomalies géo-IP, scoring de risque, prévision de demande par IA et maintenance prédictive.

Le système supporte plus de 50 utilisateurs simultanés avec provisionnement automatique de VM, latence de télémétrie robot <200ms, et journalisation d'audit inviolable avec des contrôles d'accès axés sur la sécurité. Les résultats montrent un accès sécurisé, via navigateur, aux systèmes industriels sans installation client.

Mots-clés: Industrie 4.0, IoT, Accès à Distance, Proxmox VE, Apache Guacamole, MQTT, Sécurité Zero-Trust, Maintenance Prédictive, Apprentissage Automatique